

STeM: Symbolic Tetrad Manipulator. Part I

J.Carminati¹, K. T. Vu²

¹School of Information Technology, Deakin University, Australia

²FIS, Australasia, Melbourne, Australia

October 12, 2025

Abstract

In this first part, of a two-part presentation, we present and describe, with illustrative examples, some of the basic functionality of the MAPLE computer algebra package *STeM*, which has a comprehensive collection of routines to allow the user to carry out most computations in the Newman-Penrose, Ellis-MacCallum, and Geroch-Held-Penrose formalisms, or any other tetrad formalism which is a modification of any one of these three systems.

1 Program summary

Program Title: STeMV1R1

CPC Library link to program files:

Developer's repository link: <https://github.com/jcarm1930/STeMV1R1>

Licensing provisions: CC BY-ND

Programming language: MAPLE internal language

Nature of Problem: The tetrad formalism is an approach to general relativity that generalizes the choice of basis for the tangent bundle from a coordinate basis to a more general choice of a local basis, i.e. a locally defined set of four linearly independent vector fields called a tetrad.²¹ Usually the choice of the tetrad is guided by the geometric/physical properties of the problem at hand. The advantage of the tetrad formalism over the standard coordinate-based approach to general relativity lies in the ability to choose the tetrad basis to reflect important physical/geometric aspects of the spacetime. This can be quite useful when investigating solutions and their properties, of the Einstein field equations. In the tetrad formalism, all tensors are represented in terms of their components (scalars) in a chosen basis and computations are conveniently carried out in terms of these components. Since the tensorial equations are written in terms of their basis components, large expressions can arise. This can be an advantage as well as a drawback but in many cases, however, the properties that one seeks or assumes usually become more evident. Apart from the direct use of the Ricci identities, Einstein field equations and Bianchi identities, one usually extracts important additional information through the use of the commutator conditions present in all the three major formalisms, Newman-Penrose(NP),¹⁷ Ellis-MacCallum(EM)^{10 15} and Geroch-Held-Penrose(GHP),¹² mainly used in General Relativity. Usually, the tetrad computations and resulting scalar equations, can be extensive and complicated, and well beyond hand computation ability.

Solution method: *STeM*, which runs under Maple, consists of a comprehensive collection of routines to allow the user to carry out most complex calculations and analyses in the three formalisms NP, EM, GHP or any other tetrad formalism which is a modification of any one of these three systems.

Keywords: Computer Algebra, General Relativity, Exact solutions, Tetrad formalisms.

2 Introduction: Tetrad Formalisms

In general relativity's original inception, the field equations (EFE) were written by Einstein in tensor form in a coordinate basis. In time and with the development of more sophisticated mathematics especially with the works of E. Cartan, it was recognised that the analyses, and subsequent clarification, of geometric aspects in the theory were more easily carried out with the introduction of a noncoordinated basis. Great impetus to this new approach was given by the seminal paper by Newman and Penrose (NP)¹⁷ in 1962, where with the use of spinors and spin bases, the spin coefficient, null tetrad approach was basically born. In their approach,

at each point of the spacetime, a complex null tetrad $\{\mathbf{l}, \mathbf{n}, \mathbf{m}, \bar{\mathbf{m}}\}$, with the full freedom of choice given by the 6-parameter Lorentz group, was introduced with the orthogonality properties

$$\begin{aligned} l_a l^a &= n_a n^a = m_a m^a = l_a m^a = n_a m^a = 0, \\ l_a n^a &= -m_a \bar{m}^a = 1, \end{aligned}$$

and tetrad metric

$$\eta_{ab} = \begin{bmatrix} 0 & 1 & 0 & 0 \\ 1 & 0 & 0 & 0 \\ 0 & 0 & 0 & -1 \\ 0 & 0 & -1 & 0 \end{bmatrix}$$

so that

$$g_{ab} = 2l_{(a}n_{b)} - 2m_{(a}\bar{m}_{b)}.$$

In applying the NP "method", one usually aligns the tetrad according to the symmetries of the spacetime or with the eigenvectors singled out by a particular tensor like the Weyl or Maxwell tensor with the consequence that substantial simplifications occur. The method has proven to be quite useful in a number of classification problems like those of the Weyl and Ricci tensors. We note that important solutions of the EFE were (and still are) quickly obtained, in particular the rotating Kerr solution, after the appearance of the NP formalism. This formalism is currently, more than ever, used in a wide range of applications with considerable success.

Moving on, sometime later, it was in particular noted by G. F. R. Ellis, M.A.H. MacCallum and others that the introduction of a null tetrad was not really suited to studying fluid sources and cosmological models of the EFE. Again, considerable impetus to this approach was given by the seminal papers of Ellis¹⁰ and MacCallum.¹⁵ In these spacetimes, there is a time-like unit vector $\mathbf{u}$ singled out by the fluid flow. It then seemed more appropriate to introduce an orthonormal tetrad $\{\mathbf{t}, \mathbf{x}, \mathbf{y}, \mathbf{z}\}$ at each point of the spacetime, with the orthogonality properties

$$\begin{aligned} t_a x^a &= t_a y^a = t_a z^a = x_a y^a = x_a z^a = y_a z^a = 0, \\ -t_a t^a &= x_a x^a = y_a y^a = z_a z^a = 1, \end{aligned}$$

and tetrad metric

$$\begin{bmatrix} -1 & 0 & 0 & 0 \\ 0 & 1 & 0 & 0 \\ 0 & 0 & 1 & 0 \\ 0 & 0 & 0 & 1 \end{bmatrix}$$

so that

$$g_{ab} = -t_a t_b + x_a x_b + y_a y_b + z_a z_b.$$

In applying this approach, one usually aligns the vector $\mathbf{t}$ with $\mathbf{u}$. One then projects all physical (geometric) quantities of interest, to spaces parallel and orthogonal to $\mathbf{u}$. In doing so, one obtains spacelike tensors/components like the vorticity, shear, expansion, of the fluid, and electric and magnetic parts of the Weyl tensor and so on. The tetrad is then further aligned (specialised), with the additional freedom of the SO(3) group, with these quantities, their eigenvectors, or with any symmetries present.

In 1973, R. Geroch, A. Held, and R. Penrose,¹² introduced an interesting variant of the Newman-Penrose formalism, where a pair of null directions was singled out. It is an intermediate formalism between a fully covariant one and the spin-coefficient approach. The key idea is to only retain quantities in the NP formalism which are properly weighted under the spin-boost 2-parameter subgroup of the Lorentz group. Of the original 12 complex spin coefficients only 8 remain with modifications of the derivative operators and commutator conditions. A remarkable, considerable simplification occurs. This formalism has been quite useful in the study/integration of algebraically special spacetimes or with the analysis of perturbations of them.

Regarding the *STeM* package, it is made up of three sub-systems corresponding to the three formalisms mentioned above. The orthonormal implementation follows closely the basic functionality and notation as that given by N. Van den Bergh in his program called *Oframe*.²⁴ The NP part follows the functionality and notation as that given by S Czapar and R.G. McLenghan in their NP package.⁹ Finally, The GHP part which includes all of the routines in our previous package is an expansion of our previous GHP package.²⁹ For a very comprehensive list of computer algebra packages for general relativity, we refer the reader to the article *Computer algebra in gravity research* by M.A. H. MacCallum.¹⁶

3 The Maple STeM Package: Notation and Conventions

Throughout, *STeM* uses the $(-, +, +, +)$ signature with Levi-Civita tensor defined by $\eta^{0123} = (-g)^{1/2}$. The Riemann, Ricci, and Weyl tensors are defined by

$$P^a{}_{;cd} - P^a{}_{;dc} = R^a{}_{bcd}P^b, R_{ab} = R^c{}_{acb},$$

$$C_{abcd} = R_{abcd} - \frac{1}{2}(g_{ac}R_{bd} + g_{bd}R_{ac} - g_{bc}R_{ad} - g_{ad}R_{bc}) + \frac{R}{6}(g_{ac}g_{bd} - g_{ad}g_{bc}).$$

where P is an arbitrary vector field.

The electric and magnetic parts of the Weyl tensor are defined by

$$E_{ab} = C_{agbh}u^gu^h, H_{ab} = \frac{1}{2}\eta_{ae}{}^{gh}C_{ghbd}u^eu^d.$$

Given an electromagnetic field represented by a skew tensor field F_{ab} , the electric field E_a and magnetic field H_a , as measured by an observer with 4-velocity u^a are given by

$$E_a = F_{ab}u^b, H_a = \frac{1}{2}\eta_{ab}{}^{cd}u^bF_{cd}.$$

The NP form of Maxwell's equations are

$$\begin{aligned} D\Phi_1 - \bar{\delta}\Phi_0 + \kappa\Phi_2 - 2\rho\Phi_1 + (2\alpha - \pi)\Phi_0 &= x_0J_l, \\ \delta\Phi_1 - \Delta\Phi_0 + \sigma\Phi_2 - 2\tau\Phi_1 + (2\gamma - \mu)\Phi_0 &= x_0J_m, \\ D\Phi_2 - \bar{\delta}\Phi_1 + (2\epsilon - \rho)\Phi_2 - 2\pi\Phi_1 + \lambda\Phi_0 &= x_0J_{\bar{m}}, \\ \delta\Phi_2 - \Delta\Phi_1 + (2\beta - \tau)\Phi_2 - 2\mu\Phi_1 + \nu\Phi_0 &= x_0J_n. \end{aligned}$$

where x_0 stands for the numerical constant 2π , and where

$$\begin{aligned} J^a &= J_m\bar{m}^a + J_{\bar{m}}m^a - J_ln^a - J_nl^a, \\ E_a + iH_a &= \sqrt{2}\Phi_1(l_a - n_a) + \sqrt{2}(\Phi_2m_a - \Phi_0\bar{m}_a), \\ \mathbf{E} + i\mathbf{H} &= 2\Phi_1\mathbf{e}_1 + (\Phi_2 - \Phi_0)\mathbf{e}_2 + i(\Phi_0 + \Phi_2)\mathbf{e}_3, \\ \mathbf{e}_0 &= \frac{1}{\sqrt{2}}(\mathbf{l} + \mathbf{n}), \mathbf{e}_1 = \frac{1}{\sqrt{2}}(\mathbf{l} - \mathbf{n}), \mathbf{e}_2 = \frac{1}{\sqrt{2}}(\mathbf{m} + \bar{\mathbf{m}}), \mathbf{e}_3 = \frac{-i}{\sqrt{2}}(\mathbf{m} - \bar{\mathbf{m}}), \\ E_1 &= (\Phi_1 + \bar{\Phi}_1), E_2 = \frac{1}{2}(\Phi_2 + \bar{\Phi}_2 - \Phi_0 - \bar{\Phi}_0), E_3 = \frac{i}{2}(\Phi_2 - \bar{\Phi}_2 + \Phi_0 - \bar{\Phi}_0), \\ H_1 &= i(\bar{\Phi}_1 - \Phi_1), H_2 = \frac{i}{2}(\bar{\Phi}_2 - \Phi_2 + \Phi_0 - \bar{\Phi}_0), H_3 = \frac{1}{2}(\Phi_0 + \bar{\Phi}_0 + \Phi_2 + \bar{\Phi}_2). \end{aligned}$$

With geometrized units Einstein's equations read $G_{ab} = 4x_0T_{ab}$. The electromagnetic contribution to the NP components of the Ricci tensor is $\Phi_{ab} = \Phi_a\bar{\Phi}_b$, with $\Lambda = 0$.

With our signature we have $l_an^a = -1, m_a\bar{m}^a = 1$, and the Weyl and Ricci NP scalars need to change sign, to those given by NP, in order to maintain the same form for the Einstein- Maxwell equations. These quantities are explicitly given below:

$$\begin{aligned} \Psi_0 &\equiv C_{abcd}l^am^bl^cm^d, \Psi_1 \equiv C_{abcd}l^an^bl^cm^d, \\ \Psi_2 &\equiv C_{abcd}l^am^b\bar{m}^cn^d, \Psi_3 \equiv C_{abcd}l^an^b\bar{m}^cn^d, \\ \Psi_4 &\equiv C_{abcd}n^a\bar{m}^bn^c\bar{m}^d, \end{aligned}$$

and

$$\begin{aligned} \Phi_{00} &\equiv \frac{1}{2}R_{ab}l^al^b, \Phi_{11} \equiv \frac{1}{4}R_{ab}(l^an^b + m^a\bar{m}^b), \Phi_{22} \equiv \frac{1}{2}R_{ab}n^an^b, \Lambda \equiv \frac{R}{24}, \\ \Phi_{01} &= \bar{\Phi}_{10} \equiv \frac{1}{2}R_{ab}l^am^b, \Phi_{02} = \bar{\Phi}_{20} \equiv \frac{1}{2}R_{ab}m^am^b, \\ \Phi_{12} &= \bar{\Phi}_{21} \equiv \frac{1}{2}R_{ab}m^an^b. \end{aligned}$$

The Maxwell and four-current NP scalars are

$$\Phi_0 \equiv F_{ab}l^a m^b = \phi_{11}, \Phi_1 \equiv \frac{1}{2}F_{ab}(l^a n^b + \bar{m}^a m^b) = \phi_{01}, \Phi_2 \equiv F_{ab}\bar{m}^a n^b = \phi_{00},$$

$$J_l \equiv J_a l^a, J_n \equiv J_a n^a, J_m \equiv J_a m^a, J_{\bar{m}} \equiv J_a \bar{m}^a = \bar{J}_m.$$

The three, 2-parameter null subgroups of the Lorentz group acting on the null basis vectors $\{l, n, m, \bar{m}\}$ are given below.

Null rotation leaving l^a fixed:

$$\begin{aligned}\tilde{l}^a &= l^a, \\ \tilde{n}^a &= n^a + d\bar{m}^a + \bar{d}m^a + d\bar{d}l^a, \\ \tilde{m}^a &= m^a + dl^a.\end{aligned}$$

Null rotation leaving n^a fixed:

$$\begin{aligned}\tilde{n}^a &= n^a, \\ \tilde{l}^a &= l^a + f\bar{m}^a + \bar{f}m^a + f\bar{f}n^a, \\ \tilde{m}^a &= m^a + fn^a.\end{aligned}$$

Null rotation leaving directions of l^a and n^a fixed

$$\begin{aligned}\tilde{l}^a &= F^{-1}l^a, \\ \tilde{n}^a &= Fn^a, \\ \tilde{m}^a &= e^{i\theta}m^a.\end{aligned}$$

The notation used for each formalism is given in the tables below.

Newman-Penrose Formalism: Notation

Original	STeM	Original	STeM	Original	STeM
D	D	Φ_{11}	$R11$	Φ_0	$F0$
Δ	V	Φ_{22}	$R22$	Φ_1	$F1$
δ	X	κ	k	Φ_2	$F2$
$\bar{\delta}$	Y	σ	s	V^1	Kl
Ψ_0	$W0$	ρ	r	V^2	Kn
Ψ_1	$W1$	τ	t	V^3	Km
Ψ_2	$W2$	ν	n	ψ	psi
Ψ_3	$W3$	λ	l	ϕ_{00}	$B00$
Ψ_4	$W4$	μ	m	ϕ_{10}	$B10$
Λ	L	π	p	ϕ_{11}	$B11$
Φ_{00}	$R00$	ϵ	e	J_l	$J0$
Φ_{01}	$R01$	γ	g	J_n	$J1$
Φ_{02}	$R02$	α	a	J_m	$J2$
Φ_{12}	$R12$	β	b		

Ellis-MacCallum Formalism: Notation

Original	STeM	Original	STeM	Original	STeM
e_0	$e0$	n_{12}	$n12$	Ω_1	$O1$
e_1	$e1$	n_{13}	$n13$	Ω_2	$O2$
e_2	$e2$	n_{22}	$n22$	Ω_3	$O3$
e_3	$e3$	n_{23}	$n23$	$\theta_1 = \theta_{11}$	$H1$
a_1	$a1$	n_{33}	$n33$	$\theta_2 = \theta_{22}$	$H2$
a_2	$a2$	σ_{11}	$s11$	$\theta_3 = \theta_{33}$	$H3$
a_3	$a3$	σ_{12}	$s12$	θ	$H = H1 + H2 + H3$
$\dot{u}_1$	$u1$	σ_{13}	$s13$	μ	M
$\dot{u}_2$	$u2$	σ_{22}	$s22$	p	P
$\dot{u}_3$	$u3$	σ_{23}	$s23$	$E_{\alpha\beta}$	$E11..E33$
ω_1	$w1$	σ_{33}	$s33$	$H_{\alpha\beta}$	$M11..M33$
ω_2	$w2$	V^1	$K0$	$E_{ab;c}u^c$	$dE11..$
ω_3	$w3$	V^2	$K1$	$H_{ab;c}u^c$	$dM11..$
n_{11}	$n11$	V^3	$K2$	E_α	$E1..E3$
n_{12}	$n12$	V^4	$K3$	H_α	$B1..B3$

Geroch-Held-Penrose: Notation

Original	STeM
$\mathfrak{p}$	T
$\mathfrak{p}'$	TP
δ	E
δ'	EP

In our implementation, the NP components of the Killing (Homothetic) vector are given by²⁰

$$K^a = Km\bar{m}^a + K\bar{m}m^a - Kl n^a - Kn l^a, \quad K\bar{m} \equiv \overline{Km}$$

whereas in the orthonormal frame, the Killing vector $\mathbf{K}$ is expressed as

$$\begin{aligned} \mathbf{K} &= -K0\mathbf{e}_0 + K1\mathbf{e}_1 + K2\mathbf{e}_2 + K3\mathbf{e}_3, \\ Kl &= \frac{1}{\sqrt{2}}(K0 + K1), \quad Kn = \frac{1}{\sqrt{2}}(K0 - K1), \quad K\bar{m} = \frac{1}{\sqrt{2}}(K2 + iK3) \end{aligned}$$

and the Killing equations are written as

$$K_{a;b} = Z_{ab} + \psi g_{ab},$$

where $\psi = 0$ for a Killing vector or $\psi = \text{constant} \neq 0$ for a homothetic vector.

The associated complex self-dual bivector is

$$\begin{aligned} Z_{ab}^+ &\equiv \frac{1}{2}(Z_{ab} + iZ_{ab}^*) \\ &= 2\phi_{00}l_{[a}m_{b]} + 2\phi_{01}(l_{[a}n_{b]} - m_{[a}\bar{m}_{b]}) - 2\phi_{11}n_{[a}\bar{m}_{b]} \end{aligned}$$

A list of current *STeM* package routines, for each formalism with short descriptions is given in the tables below.

Available Commands, with Syntax, for all Formalisms¹

Command/Syntax	Description
addpair(sys)	adds and factors all distinct pair additions of polys in list sys
allddeg(sys)	displays the degree of each variable in sys
autocomm(vars)	computes all 6 basic commutators of vars
buildchain(sys,chn,R)	constructs a new chain from inclusion of sys
chaincombo(PB,sys,N,T)	generates chains from all N subsets of sys, T=maximum time allowed
chainnormal(sys,chn,vars)	reduces each poly in sys to normal forms using a chain chn
clean(list)	removes a=a or 0 from a list
clearall()	clears all remember tables
clearAssumption()	clears all assumptions made by automated processes in STeM
forgetUnusedEqn()	clears all equations not used by stemnormal during process generation
forgetUsedEqn()	clears all equations used by stemnormal during process generation
comm(oper1,oper2,expr)	computes commutator of oper1,oper2, applied to expression
conj(expr)	computes complex conjugate of expr
convertGHP2NP(expr)	converts GHP expression/equation to NP
convertNP2GHP(expr)	converts NP expression/equation to GHP
convertNP2Orth(expr)	converts NP expression/equation to orthonormal
convertOrth2NP(expr)	converts orthonormal expression/equation to NP
defineOperator($Z_{new}, Z_{new} = f(Z_{old})$)	defines new operator in terms of old ones
deglist(list)	applies allddeg to a list
eqns(m,n)	downloads m to n equations of designated formalism
findCombo(sys)	attempts to find a single "suitable" variable combination in sys
findComm(sysde)	finds and computes all possible commutator pairs in sysde
findHalfComm(sysde)	finds and computes all possible higher order commutators in sysde
findVariable(struc,sys,red)	determines possible variable changes, based on user given structure, "struc", which minimises sys according to Maple nops
findops(list)	determines elements of a list of polys which have multiple ops
forgetAssumption()	forces STeM to forget all assumptions
forgetCommutator()	forces STeM to forget computed commutators
formalism(form)	sets the formalism to ghp, np or orth environment
forgetUnusedEqn()	forces STeM to clear all unused equations generated by stemnormal
forgetUsedEqn()	forces STeM to clear all used equations generated by stemnormal
formalism(form)	sets the formalism to ghp, np or orth environment
foso	brings all terms of an eqn to one side and factors

<code>genreslist(vars,nstrt,eqax,resmax,sys)</code> ⁶	computes all resultants, in pairs, w.r.t. vars of sys
<code>getAssumption()</code>	displays all assumptions made by certain routines used
<code>getUnusedEqn()</code>	retrieves all unused equations generated by automated processes
<code>getUnusedFactor()</code>	retrieves all unused factors generated by automated processes
<code>getUsedEqn()</code>	retrieves all used equations generated by automated processes
<code>getUsedFactor()</code>	retrieves all used factors generated by automated processes
<code>groebcombo(PB,sys,N,T)</code>	generates GB's from all N subsets of sys, T=maximum time allowed
<code>groebreduce(expr,PB, GBL)</code>	reduces expr using each element of GBL
<code>indeppol(sys, nfacs, zerofacs)</code>	routine removes nonzero factors (nfacs: list), and dependant polys which have factors 'zerofacs' from sys
<code>init()</code>	clears all computation results and definitions in session
<code>isCommutable(expr)</code>	checks if a particular expression is commutable ⁵
<code>linreslist(vars,num1,maxl,maxres,list)</code>	computes resultants (length < maxres) wrt linear vars[i], starting from num1 to max.length maxl of polys in list
<code>minimisePoly(sys)</code>	
<code>nullRotation(expr/eqn,preserve=l,n,or ln)</code> ³	performs null rotation of type "class" on expr/eqn
<code>orthRotation(expr/eqn, preserve=e_i)</code> ³	performs 2-d rotation of type preserving e_i , on expr/eqn
<code>picfac(expr)</code>	selects a factor choice from ops of expr
<code>polydecomp(poly,facexp)</code>	decompose poly using factor facexp
<code>polyiso(expr,combo)</code>	isolates a combo from expr
<code>powsubs2</code>	allows multiple powsubs substitutions
<code>prime(expr/eqn)</code>	computes the prime of an expr/equation
<code>properties(var)</code>	displays zero/nonzero/complex/real/weight/prime/star of var
<code>reduceMin(eqn,sys,keep=select)</code> ⁹	reduces the Maple length of eqn w.r.t. sys using LCM
<code>reduceMinSys(sys,keep=select)</code> ⁹	reduces the Maple length of sys w.r.t. itself using term-by-term LCM
<code>remdeg(sys,var,N)</code>	removes all polys from sys with degree > N in var
<code>remfacs(sys,[fac1,fac2,...])</code>	removes factors fac1,fac2,... from each poly of sys
<code>removeNZfactor(expr)</code>	removes known nonzero factors as given in showAssumption, from expr
<code>remredun(sys)</code>	removes redundant (by numerical factors) polys from sys(polys)
<code>replaceOperator(Z_{new}, $Z_{new} = f(Z_{old})$)</code>	replaces old operator with defined new one
<code>separate(list)</code>	separates polys from des from a list
<code>sepddeg(var,N,L)</code>	selects polys of L with $0 < \deg(\text{var}) \leq N$
<code>sepeqns(L,varin,varout)</code>	selects polys of L with variable varin but not varout
<code>showAssumption()</code>	displays cumulatively all nonzero assumptions made by automated processes

showCommutator()	shows all commutators calculated by findComm
showEInfo(poly)	displays information of poly
showStatistic(sys)	displays the statistics of sys
showUnusedEqnStat()	displays statistics of unused eqns by stemnormal
showUnusedFactorStat()	displays statistics of unused factors by stemnormal
simplifySys(lista,listb)	minimises pairwise, length of each element of lista against, listb using GB ⁴
specs(sys)	displays indets, length, nops and type of sys
specsop(sys)	displays indets, length, nops and type of each op of sys
star(expr/eqn)	applies the star operation
stemCollect(expr,vars)	collects an expr w.r.t.vars list
stemHTerm(poly)	displays highest ordered term in poly
stemassume([expr1, expr2, ...]) ⁵	declares expr1, expr2,... as nonzero for automated processes
stemcomplete(sys,switch=[op1,op2,op3]) ⁸	proceeds in automated fashion, continuous computation of commutators and subsequently adjoins results to sys then normalises Stops when no new commutators can be found and system is fully normalised/reduced (see stemnormal)
stemdefine(var,weight=[p,q],prime=var1,star=var2)	defines the weight, prime and star of var in a session
stemdefoper([Z1,Z2,Z3,Z4]):	defines an ordering on the operators, from lowest to highest
stemdeforder([var1, var2,...])	defines an ordering on the variables, from lowest to highest
stemeval(expr)	same as maple eval plus complex quantity evaluation in np/ghp
steminvariant([il]) invsys=prime, star or both	establishes an environment automatically invariant under invsys imposing conditions that are invariant as indicated by type,invariant
stemiso(eqn)	solves for derivatives in an equation
stemisoh(eqn/expr)	solves eqn/expr for the highest ordered term,
stemnormal(sys,switch=[op1,op2,op3])	transforms a given system of equations into a normal form based on an ordering defined by stemdeforder and stemdefoper
stemreduce(eqn/expr,sysn)	reduces, a given eqn/expr w.r.t. sysn ¹
stemsorte(sys)	sorts a system, in a solved form with highest term on LHS, in ascending order with all polynomials listed first
stemundefine(var)	clears all defined properties of var.
subtractpair (sys)	subtracts and reduces in pairs of sys.
symtran(sys,nvars,symnvars)	performs discrete symmetry transformation on sys ⁷
weight(expr/eqn/var)	displays the weight of a given expr/eqn/var

In the tables below, we indicate where the various equation groups of the basic formalisms, are located.

Newman-Penrose Equation System

Position	Description
1..18	Ricci
19..29	Bianchi
30..33	Maxwell
34..45	Killing
46..57	First IC Killing ²
58..72	Second IC Killing ²

Ellis-MacCallum Equation System

Position	Description (Eqns)
1..16	Jacobi
17..26	Field
27..42	Bianchi
43..48	E_{ab}
49..54	H_{ab}
55..58	Contracted Bianchi
59..66	Maxwell
67..76	Killing
77..81	$E_{ab;c}u^c$
82..86	$H_{ab;c}u^c$
87..104	Basic Variables
105..135	Basic EFE/I.C.
136..148	Ext./New Variables
149..153	λ and $e_i(\lambda)$

Geroch-Held-Penrose Equation System

Position	Description (Eqns)
1..12	Ricci
13..24	Bianchi
25..28	Maxwell
29..40	Killing
41..52	First IC Killing ²
53..67	Second IC Killing ²

In the above tables we have used the following abbreviations: (i). sys=list of expressions/equations of polynomials; (ii). expr=expression; (iii). Z1,Z2,..denote operators; (iv). x1,x2,..denote variables; (v). vars is a variables list, var is one variable (vi) expr is an expression with variables and/or operators; (vii). form=np,orth, or ghp; (viii) R= Polynomial Ring (ix). N=integer. (x) PB list of polynomials for ideal (xi) GBL: Groebner basis list.(xii): zerofacs: factors that are zero.¹The given system of differential equations, *sysdeqn*, is required to be in a normal form, as given by *stemnormal*. Warning: if the given system is not in a normal form, this function might get into an infinite loop;² The integrability conditions are only for a Homothetic vector;³ The *STeM* environment for the rotations must be free of global assignments, i.e. in its generic state with original variables;⁴Groebner Bases; ⁵An expression is potentially commutable if when it is solved for the highest ordered derivative all other derivatives on r.h.s. are of lower order;⁶nstrt: poly number in sys to begin resultants; eqmax: maximum length of polys used; resmax: maximum size of resultant to retain;⁷nvars: new extra variables introduced and symnvars the associated symmetry mapping of nvars.⁸Optional entries are: "algebraic", "factor" and "immstop" for control of substitution of algebraic conditions, factors, and immediately stop option after each decision step. The user should exercising care when "cutting" output from *STeM* and then pasting it in as input, in the orthonormal formalism. There is no issue with the variables but the operators paste as "e[n]", n=0..3, which is not acceptable. These may be manually changed to "en" or done so through the use of the routine *convder*.⁹The switch "select=keep" gives the user the option of which factors to keep in the reduction process otherwise automatically the largest ordered/size factor is retained

STeM running under Maple, contains all of the standard equations of the Orthonormal, NP and GHP formalisms. It automates virtually all of the regular computational tasks present in the tetrad frames, like propagation, commutation and normalisation of differential/polynomials conditions. The main key routine in *STeM* is *stemnormal* which reduces a combined list/set of polynomial and differential conditions to a "standard" form with the order of the variables and operators as specified by the user. In addition, there are five major simplifying routines in *STeM* called *stemreduce*, *minimisePoly*, *simplifySys*, *reduceMinSys* and *stemHTerm* to minimize/simplify resulting systems of equations. Briefly, *stemreduce* reduces a list of polynomial/differential conditions to a "minimal form" with respect to another list of conditions. *minimisePoly* is a routine which attempts to simplify, in pairs, elements of a set of polynomial conditions using Groebner basis methods. It can readily handle systems of large polynomial equations. *simplifySys* applies *minimisePoly* to two separate lists

minimizing/simplifying one against the other. *reduceMinSys* (*reduceMin* is for the reduction of a single condition with respect to a list of conditions) is a routine which performs a sort of exhaustive pair-term elimination using LCM's, to minimise the Maple length (or number of terms). It is computationally very intense and is more suited to small or medium sized systems. Large systems can also be addressed but would require considerably more time to complete the normalization/reduction process. *stemHTerm* presents the leading terms in a set of polynomial, based on the user's defined variable order, and then other routines in *STeM* decide, based on these leading terms, what equations are pairwise "suitable" for elimination of leading terms and hence is a sort of "restricted" Groebner method. Again, the governing principle is based on minimising Maple length, or variable elimination or term-number reduction in polynomials. Combined with the use of standard resultants, these tools can be quite effective in reducing/simplifying polynomial systems. In the next section, we will present three illustrative applications showing how *STeM* facilitates the computations typically encountered in the three main tetrad formalisms

4 STeM Applications

In the applications that follow, we shall present the exact Maple input and some of the output that is required to achieve a particular result. For cases where intermediate steps may be large, we shall suppress the output print. The key feature that we want to emphasize is that in many problems intermediate expressions may be very large and it would be of little practical value to explicitly see them. Instead, the processes and advanced tools available in *STeM* allow the user to properly and intelligently navigate through a particular sequence of very large intermediate steps using only *STeM* auxiliary information, provided by *showStatistic*, *specs*, *maplen*, *sortl*, and other routines. Of course there are also generic Maple internal commands that can be used as well

4.1 Example 1: Type D vacuum spacetimes

In this example, we shall examine vacuum Petrov type D metrics, which are solutions of Einstein's field equations, all of which have been previously given by Kinnersley. However, it was some time later that it was shown by Czapor and McLenaghan⁹ that all of these spacetimes possess the intriguing spin coefficient conditions $\tau\bar{\tau} = \pi\bar{\pi}$ and $\rho\bar{\mu} = \bar{\rho}\mu$, in the null frame that is aligned with the principal null directions of the Weyl tensor. In their computation using the Maple program NP,⁹ these identities were established, though with considerable difficulty. They used the NP formalism with a special gauge choice of $\rho = \pi$ and then performed lengthy computations with trial-and-error attempts with various substitution/reduction steps. Clearly, this can be a difficult computational problem. We shall show, using *STeM*, how these identities can be established in a simple, virtually automated fashion and in just a few minutes. Almost all of the input and output are displayed below.

We begin with the *STeM* input to establish the appropriate GHP environment with conditions and equations

```
>read 'c:/STeMV1R1.mpl':with(STeM):formalism(ghp):eqs:=eqns(1,24):
>stemdeforder([m,p,r,t,W2]):stemdefoper([E,EP,T,TP]):
>R00:=0:R01:=0:R02:=0:R12:=0:R10:=0:R20:=0:R21:=0:R22:=0:R11:=0:W0:=0:W0_c:=0:W1:=0:
W1_c:=0:W3:=0:W3_c:=0:W4:=0:W4_c:=0:k:=0:k_c:=0:s:=0:s_c:=0:n:=0:n_c:=0:l:=0:l_c:=0:L:=0:
```

A few clarify points need to be made at this stage. Firstly, we need to bear in mind that the conjugate variables are treated as independent from the variables themselves. Secondly, it is generally best to position the curvature at a higher order than the spin coefficients. However, having said that, we have noted that there have been some (very rare) exceptional cases where the best ordering is reversed. The ordering dictates to *stemnormal* (and also other routines) a key routine in *STeM*, which variables have a higher priority for the solving/reduction processes. There is no need to include the conjugates in the ordering as they are automatically positioned adjacent to their corresponding variables. If the user wishes, the conjugates can be repositioned as desired. Finally we have taken advantage of the Goldberg-Sachs theorem which in this cases yields $\kappa = \sigma = \nu = \lambda = 0$. In any case, one need not add these conditions in as they are otherwise quickly obtained by *STeM* in the reduction/normalization process.

Next, we generate the conjugates of all the equations *eqs* adjoin them to *eqs*, and apply *stemnormal* to the combined set.

```
>ceqs:=map(conj,eqs):
>toteqs:=stemnormal([op(eqs),op(ceqs)],switch=[algebraic,factor]);
```

```
toteqs := [EP(m) = -(m-m_c)p, TP(m) = -m^2, E(m_c) = p_c(m-m_c), TP(m_c) = -m_c^2,
E(p) = T(m)-mr_c-pp_c-W2, EP(p) = -p^2,TP(p) = -(t_c+p)m, E(p_c) = -p_c^2,
EP(p_c) = T(m_c)-m_cr-pp_c-W2_c, TP(p_c) = -m_c(t_c+p_c), E(r) = (r-r_c)t,
T(r) = r^2, EP(r_c) = -t_c(r-r_c), T(r_c) = r_c^2, E(t) = t^2,
EP(t) = TP(r)+m_cr+tt_c+W2,T(t) = (t+p_c)r, E(t_c) = TP(r_c)+mr_c+tt_c+W2_c,
```

$$\begin{aligned} EP(t_c) &= t_c^2, T(t_c) = r_c(t_c+p), E(W2) = 3W2t, EP(W2) = -3W2p, \\ T(W2) &= 3W2r, TP(W2) = -3W2m, E(W2_c) = -3W2_cp_c, EP(W2_c) = 3W2_ct_c, \\ T(W2_c) &= 3W2_cr_c, TP(W2_c) = -3W2_cm_c \end{aligned}$$

We proceed, by computing all commutators initially possible with *toteqs*.

```
>allcom:=findComm(toteqs);
```

```
allcom := [E(r\_c) = -T(p\_c), EP(r) = -T(p), TP(r) = -tt\_c-T(m)+pp\_c, TP(r\_c) = -tt\_c-T(m\_c)+pp\_c,
TP(t) = -E(m), TP(t\_c) = -EP(m\_c)]
```

We can query *STeM* to see what commutators have been done.

```
>showCommutator();
```

Number of full commutators calculated is , 20

They are

```
[E, EP, W2], [E, T, W2], [E, TP, W2], [EP, T, W2], [EP, TP, W2], [T, TP, W2]
[E, EP, W2\_c], [E, T, W2\_c], [E, TP, W2\_c], [EP, T, W2\_c], [EP, TP, W2\_c], [T, TP, W2\_c]
[E, T, r]
[E, T, t]
[E, TP, m\_c]
[E, TP, p\_c]
[EP, T, r\_c]
[EP, T, t\_c]
[EP, TP, m]
[EP, TP, p]
```

Number of half commutators calculated is , 0

We note that the conjugate variable commutators are also done so that the user need not manually add them. There is also the added benefit that it checks that all results are self consistent. This would assist in catching input typing errors, if they occur. Also we have the added information that no higher order commutators, indicated by "half commutators", have been computed. It is important to always check what assumptions *STeM* has made in given computations. This information can be retrieved as follows, and is always cumulative unless cleared by the user using the *clearAssumption* command at any stage.

```
>showAssumption();
```

Non-zero factors

$W2 \neq 0$

$W2_c \neq 0$

Now it is evident that the results we are seeking do not lie at the first-order commutator level. We need to address the issue of higher order commutators. We can complete this exercise by first applying *stemnormal* to the combined system consisting of *toteqs* and *allcom*.

```
>toteqsa:=stemnormal([op(toteqs),op(allcom)],switch=[algebraic,factor]);
```

```
toteqsa:=[EP(m) = -(m-m\_c)p, TP(m) = -m^2, E(m\_c) = p\_c(m-m\_c), TP(m\_c) = -m\_c^2,
E(p) = T(m)-mr\_c-pp\_c-W2, EP(p) = -p^2, TP(p) = -(t\_c+p)m, E(p\_c) = -p\_c^2,
EP(p\_c) = T(m\_c)-m\_cr-pp\_c-W2\_c,TP(p\_c) = -m\_c(t+p\_c), E(r) = (r-r\_c)t,EP(r) = -T(p),
T(r) = r^2, TP(r) = -tt\_c-T(m)+pp\_c, E(r\_c) = -T(p\_c), EP(r\_c) = -t\_c(r-r\_c),T(r\_c) = r\_c^2,
TP(r\_c) = -tt\_c-T(m\_c)+pp\_c, E(t) = t^2, EP(t) = -T(m)+pp\_c+m\_cr+W2,T(t) = (t+p\_c)r,
TP(t) = -E(m), E(t\_c) = -T(m\_c)+pp\_c+mr\_c+W2\_c, EP(t\_c) = t\_c^2,T(t\_c) = r\_c(t\_c+p),
TP(t\_c) = -EP(m\_c), E(W2) = 3W2t, EP(W2) = -3W2p,T(W2) = 3W2r, TP(W2) = -3W2m,
E(W2\_c) = -3W2\_cp\_c, EP(W2\_c) = 3W2\_ct\_c,T(W2\_c) = 3W2\_cr\_c, TP(W2\_c) = -3W2\_cm\_c]
```

We are now in a position to compute the half-commutators.

```
>halfcomm:=findHalfComm(toteqsa):
```

```

Calculate a commutator [E,EP](m)
Commutator result : -2T(m)m+2T(m)m_c+2W2m-W2_cm-pE(m)+2 mpp_c-2m_cpp_c-m_cW2-EP(E(m))
Calculate a commutator [T,EP](m)
Commutator result : -2pT(m)+pT(m_c)+T(p)m_c-T(p)m-EP(T(m))- rpm_c
Calculate a commutator [E,TP](m)
Commutator result : -TP(E(m))-3mE(m)
Calculate a commutator [T,TP](m)
Commutator result : -mp_ct_c-pE(m)-E(m)t_c+mpp_c-m_cpp_c-tpm_c-W2m-W2_cm-2T(m)m-TP(T(m))
Calculate a commutator [EP,E](m_c)
Commutator result : 2T(m_c)m-2T(m_c)m_c-m_cW2+2W2_cm_c-E(EP(m_c))-2mpp_c-W2_cm+2m_cpp_c-
p_cEP(m_c)EP(m_c))-2mpp_c-W2_cm+2m_cpp_c-p_cEP(m_c)
Calculate a commutator [T,E](m_c)
Commutator result : T(p_c)m-T(p_c)m_c+p_cT(m)-2p_cT(m_c)-E(T(m_c))-r_cp_cm
Calculate a commutator [EP,TP](m_c)
Commutator result : -TP(EP(m_c))-3m_cEP(m_c)
Calculate a commutator [T,TP](m_c)
Commutator result : -tpm_c-p_cEP(m_c)-EP(m_c)t-mpp_c+m_cpp_c
-mp_ct_c-m_cW2-W2_cm_c-2T(m_c)m_c-TP(T(m_c))
Calculate a commutator [E,EP](p)
Commutator result : -2pT(m)+pT(m_c)+T(p)m_c-T(p)m-EP(T(m))-rpm_c
Calculate a commutator [T,EP](p)
Commutator result : -3pT(p)-EP(T(p))

```

Because of space limitations, here we have only presented about one third of the automated generated output. As expected, this routine has now generated expressions for second order derivatives. The final two steps consist of normalizing *halfcomm* with *toteqsa* and then computing the final set of commutators.

```

>toteqsb:=stemnormal([op(halfcomm),op(toteqsa)],switch=[algebraic,factor]):
>allcoma:=findComm(toteqsb);

allcoma := [t_c = (-mr_c+m_cr+pp_c)/t, t_c = -(-mr_c+m_cr+pp_c)/t, E(m) = mp_c+2mt-m_ct
, EP(m_c) = -mt_c+m_cp+2m_ct_c, T(p) = 2pr-pr_c+rt_c, T(p_c) = -p_cr+2p_cr_c+r_ct]

```

We have established the identities that we were looking for plus some additional information on the derivatives. Again, we should check for any additional assumptions made by *STeM*.

```

>showAssumption();

Non-zero factors
W2 ≠ 0
W2_c ≠ 0
Non-zero denominators
t ≠ 0
t_c ≠ 0

```

Note that we have picked up the extra assumption that $\tau \neq 0$. Presumably, to complete the investigation, one should check the $\tau = 0$ subcase. We wish to emphasize that at no stage was there any trial-and-error analysis and that all computations were carried out in an automated fashion. Ultimately, this whole computation can be carried out more efficiently and more rapidly if one uses the *stemcomplete* command as follows; all output suppressed except for the final result

```

>alleqs:=stemnormal([op(eqs),op(ceqs)],switch=[algebraic,factor]):
>alleqsa:=stemcomplete(alleqs):
>halfcomm:=findHalfComm(alleqsa):
>alleqsb:=stemnormal([op(alleqsa),op(halfcomm)],switch=[algebraic,factor]):
>fineq:=stemsorte(stemcomplete(alleqsb)):

```

We can actually know a good deal about the system *fineq*, without explicitly displaying it, by using *showStatistic*. This can be useful for when one is dealing with very large systems of large equations.

```

>showStatistic(fineq);

```

Number of equations is , 34
 Number of polynomial equations is , 2
 Number of differential equations is , 32
 Equations' lengths are
 [21, 21, 51, 34, 189, 20, 33, 74, 173, 24, 189, 20, 71, 42, 23, 174, 73, 38, 37, 76, 15, 190, 15, 190, 29, 52, 25, 26,
 25, 26, 31, 40, 39, 32]
 Equations' LHS are
 [r_c, t_c, E(m), EP(m), T(m), TP(m), E(m_c), EP(m_c), T(m_c), TP(m_c), E(p), EP(p), T(p), TP(p),
 E(p_c), EP(p_c), T(p_c), TP(p_c), E(r), EP(r), T(r), TP(r), E(t), EP(t), T(t), TP(t), E(W2), EP(W2),
 T(W2), TP(W2), E(W2_c), EP(W2_c), T(W2_c), TP(W2_c), TP(W2_c)]

The total process has determined the desired two polynomial conditions, together with 32 differential conditions, which because of the *stemsorte* command will be located at the beginning of the list of equations *fineq*.

>fineq[1];fineq[2];

$$\begin{aligned}
 r_c &= m_{cr}/m \\
 t_c &= pp_c/t
 \end{aligned}$$

However, in this case, the system, through the use of *stemcomplete*, does make more nonzero assumptions. For example *STeM* assumes the following nonzero factors

$$\begin{aligned}
 &\text{Non-zero factors} \\
 &p \neq 0 \\
 &r \neq 0 \\
 &W2 \neq 0 \\
 &p_c \neq 0 \\
 &r_c \neq 0 \\
 &W2_c \neq 0 \\
 &p + 2 t_c \neq 0 \\
 &p_c + 2 t \neq 0 \\
 &2 m_c p_c r + 2 m_c r t + 2 p p_c + 2 p p_c t + W2 p_c + W2_c t \neq 0 \\
 &2 m p r_c + 2 m r_c t_c + 2 p p_c + 2 p p_c t_c + W2 t_c + W2_c p \neq 0
 \end{aligned}$$

Similarly for nonzero assumptions made on denominators. In general the price one pays for more automation is the possible introduction of additional assumptions. These special cases need further analysis. As a final point, one can, using *stemassume*, declare that certain quantities are nonzero so that *STeM* carries out the computations assuming they are not branching decisions.

4.2 Example 2: Purely magnetic spacetimes

There are no known vacuum spacetimes with a purely magnetic Weyl tensor and so it has been conjectured that no such spacetimes exist. Necessarily, the Weyl tensor must be of Petrov I, D or O. The conjecture has been established for Type D and O but despite extensive attempts by many researchers, the proof of the conjecture or the presentation of a counter example is yet to be achieved for Petrov I. The problem is deceptively difficult. In this section, we will provide part of our proof of the conjecture, in the generic case of Type I. The complete proof with full details for all special sub-cases will be published elsewhere.

This example centres on an unusual *STeM* environment consisting of a hybrid blend of null and orthonormal formalisms. We have found that if this problem is tackled purely in the null NP frame or in the Orthonormal one, computational difficulties arise; expressions become large and unwieldy. This was easily seen with quick, short analyses using *STeM*. Part of the practical usefulness of *STeM* lies in its ability to allow the user to expeditiously "experiment" with various options composed of different formalisms and variables. Curiously, it is a hybrid choice, which we will describe below, that greatly facilitates the necessary computations. We note that for reasons of space limitations and aesthetic appeal we have chosen, via a Maple "*alias*" command to represent a complex conjugate by a preceding underscore instead of a trailing "_c". The user may wish to use other representations.

We begin with the NP environment where we initially choose the null tetrad to be the canonical one for a purely magnetic Weyl tensor of Petrov Type I, so that

$$\Psi_1 = \Psi_3 = 0, \Psi_0 = \Psi_4, \Psi_0 = -\bar{\Psi}_0, \Psi_2 = -\bar{\Psi}_2,$$

and since we have vacuum then

$$\Phi_{ab} = \Lambda = 0$$

We initialize the *STeM* environment as follows and indicate nonzero quantities.

```
>read 'c:/STeMV1R1.mpl':with(STeM):formalism(np):with(student):
>alias(_c1=c1_c,_c2=c2_c,_c3=c3_c,_c4=c4_c,_t=t_c,_k=k_c,_n=n_c,_p=p_c,
_a=a_c,_b=b_c,_g=g_c,_e=e_c,_l=l_c,_m=m_c,_s=s_c,_r=r_c):
>W1:=0: _W1:=0:W3:=0: _W3:=0: _W0:=-W0: _W4:=-W4:W4:=W0:_W2:=-W2:
>L:=0:R00:=0: R01:=0: R02:=0:R10:=0: R11:=0: R12:=0:R20:=0: R21:=0: R22:=0:
>stemassume({W0,W2}):
```

Next, we define an ordering on the spin coefficients based new variables $i1, \dots, i14$, $r1, \dots, r16$, and the curvature "compound" variables which will subsequently be introduced:

```
>stemdeforder([z1,z2,w6,w8,w9,w10,w13,w14,x9,x10,i6,i5,r6,r1,i1,r2,r3,i3,r4,i4,r5,r7,i7,r8,i8,r9,i9,r10,i10,r11,i11,r12,
i12,r13,r14,i13,i14,r15,r16]);
```

Based on extensive experience in using *STeM* over a period of more than 10 years, we have up to this point in time generally selected the "curvature type" variables to be in the highest ordered positions, which in this problem would amount to placing the w and x variables at the end of the ordering list. However, for this case it turns out that the lowest ordered positions is by far, the best. With these variables placed in the highest positions, one experiences significant issues regarding size and subsequent analysis of generated conditions. We stress that there are no hard-and-fast rules in deciding the order of the variables. This depends very much on the intricate structure of the system one is dealing with. As such we would recommend that the user should experiment a little with the ordering placed on variables before carrying out in-depth analysis on any problem. We now introduce the Ricci and Bianchi equations with complex conjugates, and combine.

```
>eins:=clean(map(eval,eqns(1,29))):ceins:=map(eval,map(conj,eins)):toteqs:=[op(eins),op(ceins)]:
```

The new operators $[E0, E1, E2, E3]$, are introduced, given an ordering, properties defined, and then replaced, by issuing the following commands:

```
>defineOperator('E0', E0=(D+V)):defineOperator('E1', E1=(D-V)):defineOperator('E2',E2=(X+Y)):
>defineOperator('E3', E3=(X-Y)):stemdefoper([E3, E2, E1, E0]):
>stemdefine(E0,type=[real]):stemdefine(E1,type=[real]):stemdefine(E2,type=[real]):stemdefine(E3,type=[
imag]):replaceOperator('D', D=(E0+E1)/2):replaceOperator('V', V=(E0-E1)/2):
>replaceOperator('X', X=(E2+E3)/2):replaceOperator('Y', Y=(E2-E3)/2):
>
```

We note that because of space limitations, we have suppressed the obvious Maple output. Curiously, we have defined the new operators with a "lopsided" factor of 2. Presumably, for mainly aesthetic reasons one would want to see the $\sqrt{2}$ introduced instead so as to mimic the correspondence with orthonormal operators as closely as possible. However, if one does this then Maple is forced to work with this irrational quantity and as a consequence, one finds that the performance is reduced.

Before proceeding further with the investigation of this problem, we would like to introduce new notions of *simple* and *compound* rotations. Now the spin coefficients $\{\kappa, \sigma, \rho, \pi, \tau, \mu, \lambda, \nu\}$ are proper weighted quantities but $\{\alpha, \beta, \gamma, \epsilon\}$ are not since under the spin-boost null rotations, $\{\tilde{l}^\mu = A^{-1}l^\mu, \tilde{n}^\mu = An^\mu, \tilde{m}^\mu = e^{i\theta}m^\mu\}$, these quantities in general contain both derivatives of A and θ in their transformations. We say that these spin coefficients involve *compound* rotations. However consider the combinations $\{\alpha + \bar{\beta}, \alpha - \bar{\beta}, \epsilon + \bar{\epsilon}, \epsilon - \bar{\epsilon}, \gamma + \bar{\gamma}, \gamma - \bar{\gamma}\}$. They transform as

$$\begin{aligned}\tilde{\alpha} + \tilde{\bar{\beta}} &= e^{-i\theta} \{\alpha + \bar{\beta} - A^{-1}\bar{\delta}A\}, \tilde{\alpha} - \tilde{\bar{\beta}} = e^{-i\theta} \{\alpha - \bar{\beta} + i\bar{\delta}\theta\}, \\ \tilde{\epsilon} + \tilde{\bar{\epsilon}} &= A^{-1}(\epsilon + \bar{\epsilon}) - A^{-2}DA, \tilde{\epsilon} - \tilde{\bar{\epsilon}} = A^{-1}(\epsilon - \bar{\epsilon}) + iA^{-1}D\theta, \\ \tilde{\gamma} + \tilde{\bar{\gamma}} &= A(\gamma + \bar{\gamma}) - A^{-2}DA, \tilde{\gamma} - \tilde{\bar{\gamma}} = A(\gamma - \bar{\gamma}) - iA\Delta\theta.\end{aligned}$$

Since the transformations of these quantities involve derivatives of A or θ but **not simultaneously** both, we say that they involve *simple* boost/rotations or are just *simple*. Therefore we would like to focus our attention on *simple* real and purely imaginary combinations. Hence we arrive at the following variable transformations:

$$\begin{aligned}r_1 &= \alpha + \bar{\beta} + \bar{\alpha} + \beta, i_1 = \bar{\alpha} + \beta - \alpha - \bar{\beta}, \\ r_2 &= \bar{\alpha} - \beta + \alpha - \bar{\beta}, i_2 = \bar{\alpha} - \beta - \alpha + \bar{\beta}, \\ r_3 &= \epsilon + \bar{\epsilon} + \gamma + \bar{\gamma}, i_3 = \epsilon - \bar{\epsilon} + \bar{\gamma} - \gamma, \\ r_4 &= \gamma + \bar{\gamma} - \epsilon - \bar{\epsilon}, i_4 = -\gamma + \bar{\gamma} - \epsilon + \bar{\epsilon}.\end{aligned}$$

Next, we focus on the remaining spin coefficients. Now $\{\rho, \mu\}$ involve only the boost, and $\{\sigma, \bar{\lambda}\}$ both have the same spin factor. So continuing with the same general idea, by not mixing the different types, at this stage yet, and taking real and purely imaginary parts, modulo 2, we arrive at the following new variables for $\rho, \mu, \sigma - \bar{\lambda}, \sigma + \bar{\lambda}$:

$$\begin{aligned} r_5 &= \rho + \bar{\mu} + \bar{\rho} + \mu, i_5 = \bar{\rho} + \mu - \rho - \bar{\mu}, \\ r_6 &= \rho - \bar{\mu} + \bar{\rho} - \mu, i_6 = \bar{\rho} - \mu - \rho + \bar{\mu}, \\ r_7 &= \sigma + \bar{\lambda} + \bar{\sigma} + \lambda, i_7 = \bar{\sigma} + \lambda - \sigma - \bar{\lambda}, \\ r_8 &= \sigma - \bar{\lambda} + \bar{\sigma} - \lambda, i_8 = \bar{\sigma} - \lambda - \sigma + \bar{\lambda}. \end{aligned}$$

This leaves the remaining spin coefficients $\{\kappa, \tau, \pi, \nu\}$. By looking at similar spin transformation properties, we should consider the basic combinations: $\{\tau + \bar{\pi}, \tau - \bar{\pi}, \kappa + \bar{\nu}, \kappa - \bar{\nu}\}$. Thus we take these complex combinations, and form their real and purely imaginary parts, modulo 2,

$$\begin{aligned} c_1 &= \kappa + \bar{\nu} + \tau + \bar{\pi}, c_2 = \tau + \bar{\pi} - \kappa - \bar{\nu}, \\ c_3 &= \bar{\nu} - \kappa + \bar{\pi} - \tau, c_4 = \bar{\pi} - \tau + \kappa - \bar{\nu}, \\ r_9 &= c_1 + \bar{c}_1, i_9 = c_1 - \bar{c}_1, \\ r_{10} &= c_2 + \bar{c}_2, i_{10} = c_2 - \bar{c}_2, \\ r_{11} &= c_3 + \bar{c}_3, i_{11} = c_3 - \bar{c}_3, \\ r_{12} &= c_4 + \bar{c}_4, i_{12} = c_4 - \bar{c}_4, \end{aligned}$$

Further, we find that it is advantageous to make the following additional changes:

$$\begin{aligned} i_5 &= (i_{13} - i_{14})/2, i_7 = (i_{13} + i_{14})/2, r_4 = (r_{14} - r_{13} + 2r_6)/4, \\ r_7 &= (r_{15} + r_{16})/2, r_8 = (r_{13} + r_{14})/2, r_5 = (r_{15} - r_{16})/2. \end{aligned}$$

Interestingly, these new variables correspond to the orthonormal combinations, modulo a numerical factor, $n_{11} + n_{22} - n_{33}$, $n_{11} - n_{22} + n_{33}$, $\theta_{11} - \theta_{22}$, $\theta_{11} - \theta_{33}$, $a_1 + n_{23}$, $a_1 - n_{23}$, similarly for the others. These combinations are significantly simpler than the corresponding NP ones. As yet, it is not clear to us how these and the other variable combinations can be found while completely within the orthonormal formalism. Our input is, with the addition of z_1 and z_2 , as follows.

```
>set1:={z1=W0+3*W2,z2=W0-3*W2}
>set2:={r1 = _a + b + a + _b, i1 = _a + b - a - _b, r2 = _a - b + a - _b, i2 = _a - b - a + _b}:
>set3:={r3 = _g+e+_e+g, i3 = _g+e-g-_e, r4 = _g-e+g-_e, i4 = _g-e-g+_e}:
>set4:={r5=_r + m + r + _m, i5=_r + m - r - _m, r6=_r - m + r - _m, i6=_r - m - r + _m}:
>set5:={ c1 = k+t+_p+_n, c2 = t-k+_p-_n, c3 = _n-k+_p-t, c4 = _p-t-_n+k}:
>set5a=map(conj,set5):
>set6:={r7 = _s+l+s+_l, i7 = _s+l-s-_l, r8 = _s-l+s-_l, i8 = _s-l-s+_l}:
>set7:={c1=(r9+i9)/2, _c1=(r9-i9)/2, c2=(r10+i10)/2, _c2=(r10-i10)/2, c3=(r11+i11)/2, _c3=(r11-i11)/2,
c4=(r12+i12)/2, _c4=(r12-i12)/2, i5=(1/2)*i13-(1/2)*i14, i7=(1/2)*i14+(1/2)*i13,
r4=(1/4)*r14-(1/4)*r13+(1/2)*r6, r8=(1/2)*r13+(1/2)*r14, r5=(1/2)*r15-(1/2)*r16, r7=(1/2)*r16+(1/2)*r15}:
Our semi-final variable set is {r1, r2, r3, r6, r9, .., r16, i1, .., i4, i6, i8, .., i14, z1, z2}, correctly yielding 24 independent real/imaginary functions. We introduce the properties of the new variables, into the session by two small "do" loops with just one entry:
```

```
>for j from 1 to 16 do stemdefine(r||j,type=['real']):od:
>for j from 1 to 14 do stemdefine(i||j,type=['imag']):od:
```

and

```
>stemdefine(z1,type=['imag']):stemdefine(z2,type=['imag']):
```

We need one final change which scales the variables $\{i6, i8, i9, i10, i13, i14, r9, r10\}$ by the curvature components and by $\{i1, i3, i4, r1\}$.

```
>set8:={i10 = w10*i1/z2, i13 = i3*w13/z1, i14 = i3*w14/z2, i6 = w6*i4/(z2*z1), i8 = w8*i4/(z2*z1),
i9 = w9*i1/z1, r10 = x10*r1/z1, r9 = r1*x9/z2}:
>stemdefine(z1,type=['imag']):stemdefine(z2,type=['imag']):stemdefine(x9,type=['imag']):stemdefine(x10,type=['imag']):
>stemdefine(w13,type=['imag']):stemdefine(w14,type=['imag']):stemdefine(w6,type=['real']):stemdefine(w8,type=['real']):
>stemdefine(w9,type=['imag']):stemdefine(w10,type=['imag']):
```

We can easily compare our new variables in terms of the standard orthonormal ones as follows, for example:

```
>factor(solve(map(convertNP2Orth,set2),{r1,i1,r2,i2}));

{i1 = I√2(s13-w2), i2 = I√2(a3-n12), r1 = √2(s12+w3), r2 = (a2+n13)√2}

>factor(solve(map(convertNP2Orth,set6),{r7,i7,r8,i8}));

{i7 = I√2(n22-n33), i8 = (2I)√2s23, r7 = 2√2n23, r8 = -(H2-H3)√2}
```

A check of all the new variables shows that they are intriguingly simple combinations of the orthonormal ones. However, curiously, we have found that, for this problem, a direct application of the standard orthonormal system leads to insurmountable difficulties.

We complete the setup of the *STeM* environment by making a global total switch from the NP standard variables to our new ones.

```
>solve({op(set1),op(set2),op(set3),op(set4),op(set5),op(set5a),op(set6)},
{a,_a,b,_b,g,_g,p,_p,t,_t,e,_e,n,_n,m,_m,k,_k,l,_l,r,_r,s,_s,W0,W2}):assign(%):
>assign(set7):assign(set8):
```

The stage is now set for the analysis of the complete set of the extended field equations, with the application of *stemnormal*. We display some of the output with decision choices.

```
>toteqsa:=stemnormal(toteqs,switch=[algebraic,factor]):

"Obtain algebraic equation"

i1*w10-i1*w9+r1*x10-r1*x9

Substitute this algebraic equation into the system? (y,n) Y
"Obtain algebraic equation"

r1*(x10-x9)

Substitute this algebraic equation into the system? (y,n) Y
factor , 1, : , r1
factor , 2, : , x10 - x9
Use factor number?(ter=terminate) 2
"Obtain algebraic equation"

i1

Substitute this algebraic equation into the system? (y,n) Y
"Obtain algebraic equation"

x10-2*z1-2*z2

Substitute this algebraic equation into the system? (y,n) Y
"Obtain algebraic equation"

x9-2*z1-2*z2

Substitute this algebraic equation into the system? (y,n) Y
"Obtain algebraic equation"

i3*w13*z1*z2+i3*w14*z1*z2+i4*w6*z1-i4*w6*z2-i4*w8*z1-i4*w8*z2

Substitute this algebraic equation into the system? (y,n) Y
"Obtain algebraic equation"

i3*(w13+w14)

Substitute this algebraic equation into the system? (y,n) Y
```


Therefore there are 24 polynomial conditions of the indicated lengths. Even though the system *totpols* is quite simple we can in fact simplify it much further, using the routines *minimisePoly* and *reduceMinSys*, as follows. Before that, we should query *STeM* to indicate what assumptions have been made.

```
>showAssumption();
```

```
Non-zero factors
i3 ≠ 0
r1 ≠ 0
Non-zero denominators
r1 ≠ 0
w6 ≠ 0
w8 ≠ 0
z1 ≠ 0
z2 ≠ 0
w6 - w8 ≠ 0
-z1 - z2 ≠ 0
-w10 + w9 ≠ 0
-w6 z1 + w6 z2 + w8 z1 + w8 z2 ≠ 0
```

```
>totpolb:=minimisePoly(totpols):
>totpolc:=reduceMinSys(totpolb):
>findops(totpolc);
```

11, 12

```
>totpolc[11]:=totpolc[11]/z1/z2;
```

```
totpolc[11] := 3*i11*r1*z2-3*i12*r1*z1-6*i2*r1*z1+6*i2*r1*z2-6*i3*r14*z1+2*z1^2-8*z1*z2
```

```
>totpolc[12]:=totpolc[12]/(x9-2*z1);
```

```
totpolc[12] := i11*r1*x9+2*i11*r1*z1-2*i12*r1*z1+2*i2*r1*x9-2*i3*r13*z1-i3*r14*x9-2*i3*r14*z1
```

```
>showStatistic(totpolc);
```

```
Number of equations is , 13
Number of polynomial equations is , 13
Number of differential equations is , 0
Equations' length are
[2, 2, 12, 12, 12, 16, 92, 104, 104, 115, 136, 140, 167]
Equations' LHS are
[none, none, none, none, none, none, none, none, none, none,
none, none, none]
```

We see that the system of polynomials has been reduced considerably. Finally, we use the routine *PolynomialSystem*, in Maple, to solve *totpolc*.

```
> with(SolveTools):
>sola:=PolynomialSystem(totpolc,{z1<>0,z2<>0});
```

```
sola := {i1 = 0, i11 = i11, i12 = i12, i2 = i2, i3 = i3, i4 = 0, r1 = 2z1/(2i2+i12), r13 = -2z1/i3,
r14 = -2z1(-i12+i11)/(i3(2i2+i12)), w13 = 2z1, w14 = -2z1, x10 = -2z1, x9 = -2z1, z1 = z1, z2 = -2z1}
```

Note that $2i_2+i_{12} \neq 0$ since $z_1 z_2 \neq 0$. There is a solution to the conditions obtained so we are not quite done. To finish this example, we simply apply *E0* to $z_2 = -2z_1$.

```
>fsubs(sola,stemreduce(E0(z2=-2*z1),toteqsa));
```

$-12z_1^2/i_3$

Thus the proof for this generic case is complete. We will stop here. There are of course, a number of special cases, that need to be analyzed, as given by $r_1 i_3 w_6 w_8 (w_6 - w_8)(w_9 - w_{10})(-w_6 z_1 + w_6 z_2 + w_8 z_1 + w_8 z_2) = 0$. It turns out that some of these subcases are more involved. However, the overall resulting systems, that we do find, have the desirable feature that there occurs a good measure of variable separation in the constituent conditions which assists the analysis considerably, though, admittedly, there results a large number of branching cases.

It is apparent from the above work, that it was not necessary, at any stage, to explicitly display the complete system. There are a number of routines in *STeM* which assist the user to extract essentially a great deal of the important/relevant information, of any given system without displaying it.

As a final point, it is conceivable that the new variables/system introduced here may prove to be useful for other applications. This matter is currently under investigation.

4.3 Example 3: Shear-free perfect fluids

We now present this last application as an interesting demonstration of a quite difficult problem made relatively straightforward by *STeM*. General relativistic, shear-free, perfect fluids which obey a barotropic equation of state $p = p(\mu)$ (μ is the energy density and p is the pressure) such that $\mu + p \neq 0$ have been studied extensively over the past fifty years. Much effort, by numerous researchers, has gone into showing that such fluids must be either nonexpanding (the expansion scalar $\theta = 0$) or nonrotating (the vorticity scalar $\omega = 0$). (The reader is referred to the review by Collins⁵ and Van den Bergh and Slobodeanu²⁶ for comprehensive discussions of shear-free fluids and the associated conjecture.) Indeed, the conjecture has been demonstrated to hold in numerous special cases such as:

- spatial homogeneity,^{1,13} the cases where $p' = 0, \pm 1/3, 1/9$,^{6,8,11,23,25}
- the case where the vorticity ω^a is parallel to the acceleration $\dot{u}^a$,^{18,30}
- the case when $\theta = \theta(\mu)$,¹⁴
- the case when $\theta = \theta(\omega)$,¹⁹
- purely electric⁴ or purely magnetic⁸ spacetimes,
- Petrov Type O, Petrov Type N,² or Type III³ spacetimes.

The conjecture has been also studied by Van den Bergh and Slobodeanu(VdBS)²⁶ for a linear equation of state and Van den Bergh for the case where $\mathfrak{D}_1 = \mathfrak{R}_2 = 0$, with a Killing vector.²⁷(see Appendix). In both cases the conjecture was established. A summary of their proofs will now be given.

The starting point is the setting up of the EFE and the alignment of the tetrad, as follows. The Einstein field equations are

$$R_{ab} - \frac{1}{2}Rg_{ab} = \mu u_a u_b + p h_{ab}, \quad (1)$$

with $p = p(\mu)$, where $\mathbf{u}$ is the future-pointing (time-like) unit tangent vector to the flow, and $h_{ab} = g_{ab} + u_a u_b$ is the projection tensor into the rest space of the observers with 4-velocity $\mathbf{u}$. Next, $\mathbf{e}_0$ and $\mathbf{e}_3$ are aligned with $\mathbf{u}$ and ω , respectively, such that $\omega = \omega \mathbf{e}_3 \neq 0$. The relevant variables then become $\mu, p, \theta, \omega, \dot{u}_\alpha, \Omega_\alpha, \sigma_{\alpha\beta} = 0$, together with the quantities $n_{\alpha\beta}$ and a_α . Latin indices will be spacetime indices while Greek indices will take the values 1, 2 and 3.

Secondly, it is always possible, using the remaining group freedom and making use of the Jacobi identities and field equations, to further specialize the tetrad so as to achieve

$$\Omega_1 = \Omega_2 = \Omega_3 + \omega = 0, \quad (2)$$

$$n_{11} = n_{22} \equiv n. \quad (3)$$

Now, if $n_{12} \neq 0$, the tetrad is completely fixed. On the other hand, if $n_{12} = 0$ ($\mathfrak{D}_3 = 0$) then there exists additional gauge freedom of a *basic* rotation in the $\langle \mathbf{e}_1, \mathbf{e}_2 \rangle$ plane. Next, VdBS replace $n_{\alpha\beta}$ and a_α with the new variables q_α and r_α , defined by $n_{\alpha-1\alpha+1} = (r_\alpha + q_\alpha)/2$, and $a_\alpha = (r_\alpha - q_\alpha)/2$, expressions which must be read modulo 3. In addition, new extension variables, z_a , defined by

$$\partial_0 \theta = -\theta^2/3 + 2\omega^2 - (\mu + 3p)/2 + j, \quad \partial_0 \theta = z_0, \partial_1 \theta = z_1, \partial_2 \theta = z_2, \partial_3 \theta = z_3, \quad (4)$$

are also introduced.

As regards the proof for the linear equation of state, it relies substantially on *basic* functions, which are functions f that are conserved along the flow of the fluid i.e. $\mathbf{e}_0(f) = 0$. In this case, VdBS, after some initial general results are established, introduce the following variables

$$\lambda \equiv \exp\left(\int \frac{d\mu}{3(p+\mu)}\right), p' = \frac{r}{3}-1, p = \left(\frac{r}{3}-1\right)\lambda^r - \mu_0, \mu = \lambda^r + \mu_0$$

(μ_0, r constants, since p is linear) The proof is essentially composed of two parts. The first part consists of showing that either $\dot{U}_3 \equiv \dot{u}_3/(\lambda p')$, is basic, in which case they are done by their theorem 2 or that with further analysis there exists a Killing vector parallel to the vorticity. Further, if $\theta\omega \neq 0$, they obtain

$$\dot{u}_3 = z_3 = q_3 = r_3 = E_{13} = E_{23} = 0, \quad (5)$$

together with an additional condition on E_{33} . In this case $n_{12} = 0$, and they use the additional basic rotation freedom to set $\mathbf{b}_2 = \mathbf{b}_1$.

Next, they translate these conditions into their basic variables equivalent: $\mathbf{b}_3 = \mathfrak{D}_3 = \mathfrak{R}_3 = \mathfrak{C}_{13} = \mathfrak{C}_{23} = 0$ and start deriving conditions from the field equations and first and second Bianchi identities. There follows a long analysis (with $p' \neq -1/11, 1/4$) of conditions involving the full set of basic variables and their extensions, $\dot{U}_\alpha$, and multiple time-like propagations (their repeated time-propagations should be on equation (80) not as indicated equation (79)) with resultants, to obtain three relations of the form

$$\mathcal{P}_i = a_i U^2 + b_i U \theta^2 + c_i U + d_i \theta^2 + e_i, \quad i = 1, 2, 3.$$

a_i, b_i, c_i, d_i , and e_i have basic coefficients of degree 2 and 20 in respectively $p + \mu$ and λ , and $U = \dot{U}_1^2 + \dot{U}_2^2$, where $\dot{U}_\alpha \equiv \dot{u}_\alpha/(\lambda p')$. VdBS then eliminate θ , by resultants to obtain two relations of the form $\mathcal{R}_i(U, p + \mu, \lambda, \mu) = 0$, $i = 1, 2$. (We note a minor correction that the $\mathcal{R}_i$ must be a function of μ and the constant μ_0 in addition to the $\mu + p$ combination with U and λ). The $\mathcal{R}_i$ are of degree 3 in U , and of degree 9 in $p + \mu$ and 30 in λ . Taking one final resultant with respect to U yields $\mathcal{F}$, which factors as

$$\begin{aligned} \mathcal{F} = & \mathfrak{o} p'^{13} (p + \mu)^{18} \lambda^{18} (4p' - 1)(6p' + 1)(3p' - 1)^9 (21p' + 11)^4 \\ & \times (11p' + 1)^6 (117p'^2 + 69p' + 2)(96p'^2 + 47p' + 1) \mathcal{F}_1 \mathcal{F}_2 \end{aligned}$$

which must vanish for consistency. $\mathcal{F}_1$ and $\mathcal{F}_2$ respectively of degrees (6,20) and (21,70) in $p + \mu$ and λ , with coefficients depending on $\mathfrak{o}, \mathfrak{J}, \mathbf{b}_1, \mathfrak{C}_3, \mathfrak{X}_3$. Presumably, these polynomials must be relatively large though the authors do not give any indication of the overall (Maple or Mathematica) size. They then argue that any such polynomial may be written in the form $\sum_{i,j} c_{i,j} \lambda^{ir+j} = 0$, where $c_{i,j}$ are functions of the essential indicated basic variables and one extension variable (see their Appendix B). We note that they next need to consider various possible values of rational and irrational values of r in combination with multiple applications of resultants. These considerations result in rather lengthy complex arguments which will not be summarized here due to space limitations. Eventually, the authors arrive at a number of separate case conclusions that all lead to $\theta\omega = 0$. They indicate that full details, in Maple or Mathematica worksheets are available, since they have only described the outline of the proof; the output being too lengthy for publication.

Using *STeM*, we have been able to construct a proof of the gamma law case which does not rely on any complicated arguments concerning rational/irrational powers of (possibly complex) λ or r , and relies substantially on much smaller equations. The full details of this new approach will be published elsewhere.

For the proof of the conjecture for the case when $\kappa \equiv \mathfrak{D}_1 - i\mathfrak{R}_2 = 0$, general $p = p(\mu)$ with a Killing vector parallel to the vorticity of the fluid, Van den Bergh²⁷ utilizes a newly developed complex formalism, which is somewhat reminiscent of the GHP variant of NP, of the reduced orthonormal system with basic variables. Briefly, the complex scaled fluid velocity is defined by $\dot{u} = \dot{U}_1 + i\dot{U}_2$. The quantities $\kappa, \pi, \rho, \beta, \mathcal{E}_1, \mathcal{E}_2, \mathcal{E}$, which are simple complex combinations of the basic variables together with $\mathcal{O} \equiv \mathfrak{o}$ and $\mathcal{J}$, and which are well weighted under $\text{SO}(2)$, appear explicitly in the equations whereas the quantities τ and ν which are not well weighted do not appear explicitly but form part of the well weighted operators $\mathfrak{D}, \mathfrak{D}'$ and $\mathcal{Z}$. Finally after some recombining, there results a complex system consisting of field equations with integrability conditions. The conditions $\kappa = 0$ and the existence of the Killing vector are then explicitly introduced with some immediately resulting simple conditions. Next he considers two separate cases based on whether $6p'G + 3p' + 1$ is zero or not. In the first case, he obtains a real equation for $|\dot{u}|^2$. Time propagating this condition and combining with other equations leads to a relatively large equation of the form

$$\lambda \bar{\beta} \dot{u} = -iQ_1 \epsilon_0 \theta \mathcal{J} + iQ_2 \mathcal{O}^2 \theta / \epsilon_0 + Q_3 \mathcal{O} \mathcal{J} \lambda + Q_4 \mathfrak{B} \mathcal{O} \lambda + Q_5 \mathcal{O} \mathcal{J} \lambda + Q_6 \mathfrak{B} \mathcal{O} \lambda + Q_7 \lambda^3 \epsilon_0 \mathcal{O} + Q_8 \lambda \mathcal{O}^3 / \epsilon_0^2$$

where Q_i are rational functions of p' and coefficients are basic including $\mathfrak{B}$ (curiously, $\mathfrak{B}$ which is an extension or auxiliary variable, is never explicitly determined only the integrability condition that it satisfies). The key

feature of this equation is that θ is present only in its imaginary part. Taking the real part of this condition and combining with multiple time propagations of resulting equations leads to, after elimination of some variables presumably by resultants, a linear equation of state and hence the proof is complete for this branch. In the case $6p'G + 3p' + 1 \neq 0$, the situation is considerably more complicated. He begins with solving a large integrability equation for $\dot{u}$, which when combined with the time propagation of the same equation leads to a relatively lengthy equation of the form

$$C|\dot{u}|^2 + H_1\mathcal{J} + H_2\mathfrak{B} + H_3\mathcal{O}^2/\epsilon_0^2 + H_4\epsilon_0\lambda^2 = 0$$

where H_i are rational functions of G and p' , C is a rational function of μ, p, p', G and G' . Firstly, he notes that if $C = 0$, then combining with two time propagations of the above equation and some variable elimination there results a condition of the form $P_1\mathcal{O}^2 + P_2 = 0$, where P_1 and P_2 are, presumably (the actual sizes are not stated) large polynomial functions, of degree 9 in G and of degree 20 and 17 in p' , respectively. Finally, a resultant (presumably large hence with a mod) is used between P_1 and P_2 to eliminate G . Since the resultant does not vanish identically, it follows that the equation of state is linear and the proof is complete, in this case. Finally, for $C \neq 0$, then after some propagations with recombining of equations, he obtains an equation of the form

$$\mathfrak{B}\mathcal{F}_1 + \mathcal{J}\mathcal{F}_2 + \mathcal{O}^2\mathcal{F}_3 + \mathcal{F}_4 = 0,$$

where $\mathcal{F}_i$ are somewhat lengthy rational functions of $\lambda, \epsilon_0, \mu, p, p', G$, and G' . At this stage lemma 1 is invoked which is just basically a separation-of-variables argument focusing on the, constant coefficient linear independence of the set $\{\mathfrak{B}, \mathcal{J}, \mathcal{O}^2, 1\}$. It follows that there are five distinct separate cases to consider delineated by the zero/nonzero or constant ratio properties of $\{\mathcal{F}_1, \mathcal{F}_2, \mathcal{F}_3, \mathcal{F}_4\}$. After a very lengthy and tedious computation, it is eventually shown that all possible cases lead to a linear equation of state. We have also examined this case and using *STeM*, have obtained a relatively short concise proof which does not require any complicated arguments. In this our final example, we shall present our version of the proof of the conjecture when $\kappa = 0$ and there exists the indicated Killing vector. We shall suppress much of the output generated by *STeM* (it is not necessary to see it) but display only statistically important information with key conditions, which are relatively small. We shall argue by contradiction, firstly assuming $\theta\omega \neq 0$, and then showing that either $\theta = 0$ or $\omega = 0$ must follow.

4.3.1 $\kappa = 0$ with Killing vector parallel to vorticity

We begin our analysis by setting up initially the *STeM* environment, the same as that of VdBS, which includes the familiar start-up input for the EFE and tetrad alignment as described above. We note that the presence of the Killing vector makes all propagations along the vorticity direction vanish

```
>read 'c:/STeMV1R1.mpl';with(STeM):with(student):
```

STeM_v1r1 (May-2025)(c), by K. T. Vu and J. Carminati

The authors would kindly request that this software be referenced if it is used in work eventuating in a publication

Users are free to modify this program but it is not to be redistributed in modified form

```
>stemdeforder([x,p,mu,theta,omega,P1,P2,P3,P4,P5,K3,U1,U2,U3,J,b1,b2,b3,C0,C3,D1,D2,D3,R,R1,R2,R3,C12,C13,
C23,seq(X[i],i=1..16),u1,u2,u3,z1,z2,z3,q1,q2,q3,r1,r2,r3,n,n33,E11,E22,E12,E13,E23,M11,M22,M12,M13,M23,j]):
>formalism(orth):stemassume(P1,P+mu,P2,omega,theta,P,P1+1,P1-1,mu):
>H:=theta:P:=p:M:=mu:W:=omega:stemdefoper([e3,e2,e1,e0]):
>e0(P):=P1*e0(M):e1(P):=P1*e1(M):e2(P):=P1*e2(M):e3(P):=P1*e3(M):
>e0(P1):=P2*e0(M):e1(P1):=P2*e1(M):e2(P1):=P2*e2(M):e3(P1):=P2*e3(M):
>e0(P2):=P3*e0(M):e1(P2):=P3*e1(M):e2(P2):=P3*e2(M):e3(P2):=P3*e3(M):
>e0(P3):=P4*e0(M):e1(P3):=P4*e1(M):e2(P3):=P4*e2(M):e3(P3):=P4*e3(M):
>e0(P4):=P5*e0(M):e1(P4):=P5*e1(M):e2(P4):=P5*e2(M):e3(P4):=P5*e3(M):
>assign(eqns(140,145)):E33:=-E22-E11:M33:=-M22-M11:O1:=0:O2:=0:
>H:=theta:P:=p:M:=mu:H1:=H/3:H2:=H/3:H3:=H/3:s12:=0:s23:=0:s13:=0:
>w1:=0:w2:=0:w3:=omega:O3:=-omega:n22:=n:n11:=n:K0:=0:K1:=0:K2:=0:psi:=0:
```

We proceed by normalising the complete set of equations and assigning the determined variables.

```
>hqa:=stemnormal([op(eqns(1,58)),op(eqns(136,139)),op(eqns(67,76))],switch=[factor,algebraic]):
```

factor , 1, : , K3

factor , 2, : , u3

Use factor number?(ter=terminate) 2

"Obtain algebraic equation"

$$-3*n33*\omega+2*z3$$

Substitute this algebraic equation into the system? (y,n) y
"Obtain algebraic equation"

$$-2*\omega^2+3*q1*u1-3*r2*u2+3*E11+3*E22-j$$

Substitute this algebraic equation into the system? (y,n) y
"Obtain algebraic equation"

$$3*E23*\omega+u1*z3$$

Substitute this algebraic equation into the system? (y,n) y
"Obtain algebraic equation"

$$3*E13*\omega-u2*z3$$

Substitute this algebraic equation into the system? (y,n) y
"Obtain algebraic equation"

$$3*\omega*r2-3*M23-z1$$

Substitute this algebraic equation into the system? (y,n) y
"Obtain algebraic equation"

$$3*\omega*q1+3*M13-z2$$

Substitute this algebraic equation into the system? (y,n) y
"Obtain algebraic equation"

$$z3+3*M12$$

Substitute this algebraic equation into the system? (y,n) y
"Obtain algebraic equation"

$$z3$$

Substitute this algebraic equation into the system? (y,n) y
"Obtain algebraic equation"

$$n33$$

Substitute this algebraic equation into the system? (y,n) y
"Obtain algebraic equation"

$$E13$$

Substitute this algebraic equation into the system? (y,n) y
"Obtain algebraic equation"

$$E23$$

Substitute this algebraic equation into the system? (y,n) y
"Obtain algebraic equation"

$$M12$$

Substitute this algebraic equation into the system? (y,n) y
"Obtain algebraic equation"

$$M11$$

Substitute this algebraic equation into the system? (y,n) y
"Obtain algebraic equation"

$$M22$$

Substitute this algebraic equation into the system? (y,n) y

```
>solpol:=solve([op(getUsedEqn()),op(getUsedFactor())],{E13,E23,M11,M12,M22,j,n33,u3,M23,M13,z3});
```

$$\text{solpol} := \{E13 = 0, E23 = 0, M11 = 0, M12 = 0, M13 = -\omega q_1 + \frac{1}{3}z_2, M22 = 0, M23 = \omega r_2 - \frac{1}{3}z_1, \\ j = -2\omega^2 + 3q_1u_1 - 3r_2u_2 + 3E11 + 3E22, n33 = 0, u3 = 0, z3 = 0\}$$

```
>assign(solpol);
```

```
>specs(hqa);
```

```
indets = {E11, E12, E22, K3, P1,  $\mu$ , n,  $\omega$ , p, q1, q2, q3, r1, r2, r3,  $\theta$ , u1, u2, z1, z2, e0(E11), e0(E12), e0(E22),
e0(K3), e0( $\mu$ ), e0(n), e0( $\omega$ ), e0(q1), e0(q2), e0(r1), e0(r2), e0( $\theta$ ), e0(z1), e0(z2), e1(E11), e1(E12), e1(E22),
e1(K3), e1( $\mu$ ), e1(n), e1( $\omega$ ), e1(q1), e1(r1), e1(r2), e1( $\theta$ ), e1(u1), e1(u2), e1(z1), e1(z2), e2(E11), e2(E12),
e2(E22), e2(K3), e2( $\mu$ ), e2(n), e2( $\omega$ ), e2(q1), e2(q2), e2(r1), e2(r2), e2( $\theta$ ), e2(u1), e2(u2), e2(z1), e2(z2), e3(E11),
e3(E12), e3(E22), e3(K3), e3( $\mu$ ), e3( $\omega$ ), e3(q1), e3(q2), e3(r1), e3(r2), e3( $\theta$ ), e3(u1), e3(u2), e3(z1), e3(z2)}
length = 4280
nops = 59
type = list
```

```
>showAssumption();
```

Non-zero assumption predefined by user

$P1 \neq 0$

Non-zero factors

$K3 \neq 0$

Non-zero denominators

$\omega \neq 0$

At this stage, we determine all possible commutators using the automated internal routine in *STeM*, and then separate and collect the resulting conditions. It is important to keep the polynomial conditions separate from the differential ones.

```
>clearall();allcoma:=findComm(hqa):
```

```
>specs(%);
```

```
indets = {P1, P2,  $\mu$ , n,  $\omega$ , p, q1, q2, r1, r2,  $\theta$ , u1, u2, z1, z2, e0( $\omega$ ), e0(u1), e0(u2), e1(z1), e2(z2), e3(z1), e3(z2)}
length = 628
nops = 7
type = list
```

```
>allcomas:=separate(allcoma):
```

```
>maplen(allcomas[1]);
```

[]

```
>showAssumption();
```

Non-zero factors

$K3 \neq 0$

$P1 \neq 0$

$\mu + p \neq 0$

Non-zero denominators

$P1 \neq 0$

$u1 \neq 0$

Next, we update and normalise the augmented set of differential conditions followed by a subsequent determination of additional commutator conditions and repeat the above process.

```
>hqb:=stemnormal([op(hqa),op(allcomas[2])],switch=[algebraic,factor]):
```

```
>allcombs:=separate(findComm(hqb)):maplen(allcombs[1]);
```

[]

```
>maplen(allcombs[2]);
```

[256, 256, 256, 256, 951, 1036, 1155]

```
>showAssumption();
```

Number of full commutators calculated is , 48

They are

```
[e1, e0, K3], [e2, e0, K3], [e2, e1, K3], [e3, e0, K3], [e3, e1, K3], [e3, e2, K3] [e1, e0, μ], [e2, e0, μ],
[e2, e1, μ], [e3, e0, μ], [e3, e1, μ], [e3, e2, μ] [e1, e0, ω], [e2, e0, ω], [e2, e1, ω], [e3, e0, ω], [e3, e1, ω],
[e3, e2, ω] [e1, e0, q1], [e2, e0, q1], [e2, e1, q1], [e3, e0, q1], [e3, e1, q1], [e3, e2, q1] [e1, e0, r2],
[e2, e0, r2], [e2, e1, r2], [e3, e0, r2], [e3, e1, r2], [e3, e2, r2] [e1, e0, θ], [e2, e0, θ], [e2, e1, θ],
[e3, e0, θ], [e3, e1, θ], [e3, e2, θ] [e1, e0, u1], [e2, e0, u1], [e2, e1, u1], [e3, e0, u1], [e3, e1, u1],
[e3, e2, u1] [e1, e0, u2], [e2, e0, u2], [e2, e1, u2], [e3, e0, u2], [e3, e1, u2], [e3, e2, u2]
```

Number of half commutators calculated is , 0

```
>showAssumption();
```

Non-zero factors

$K3 \neq 0$

$P1 \neq 0$

$\mu + p \neq 0$

Non-zero denominators

$P1 \neq 0$

$u1 \neq 0$

$\omega \neq 0$

$3 P1 + 1 \neq 0$

```
>hqc:=stemnormal([op(hqb),op(allcombs[2])],switch=[algebraic,factor]):
```

Obtain algebraic equation with length = 1341

and containing {E11, E22, P1, P2, P3, mu, omega, p, q1, r2, theta, u1, u2, z1, z2}

Substitute this algebraic equation into the system? (y,n) n

```
>pola:=[op(getUnusedEqn())]:showAssumption();
```

Non-zero factors

$K3 \neq 0$

$P1 \neq 0$

$\mu + p \neq 0$

Non-zero denominators

$P1 \neq 0$

$u1 \neq 0$

$\omega \neq 0$

$3 P1 + 1 \neq 0$

It is best, at this stage, to manually carry out the indicated commutators below using the differential system *hqc*. A simple check shows that the automated process using *findComm* would introduce too many unnecessary nonzero assumptions.

```
>comvars:=[K3,M,n,W,q1,r2,H,u1,u2,z1,z2]:
```

```
>for i from 1 to 11 do e3(comvars[i]):=0;od:
```

```
>e3(q2):=0:e3(r1):=0:clearall();
```

```
>comall:=stemreduce(map(autocomm,comvars),hqc):
```

```
>clean([op(comall[1]),op(comall[2])]);
```

$[-K3nr2 = 0, -K3nq1 = 0, -nu2(\mu+p)/P1 = 0, nu1(\mu+p)/P1 = 0]$

```
>com03s:=clean(stemreduce([comm(e3,e0,E11),comm(e3,e0,E22),comm(e3,e0,E12)],hqc)):
```

```
>n:=0:comres:=sort1(clean([seq(op(comall[i]),i=1..11),op(com03s)]))):
```

```
>specs(comres);
```

indets = {E11, E12, E22, P1, P2, P3, P4, μ , ω , p, q1, q2, r1, r2, θ , u1, u2, z1, z2}

length = 87506

nops = 7

type = list

```
>clearall():hqc:=stemnormal(hqc,switch=[algebraic,factor]):
```

```
>comhqcs:=separate(clean([op(halfcomhqcs)])):maplen(%):
```


□

```
>maplen(clean(comhqcs[2]));
```

[17, 17, 1153, 1153, 1299, 12563, 16321, 17411, 18213, 18842, 20217]

The differential system *hqc* is augmented with the newly derived differential conditions and then normalised to give *hqd*. The complete set of polynomials is stored in *addpolsa*.

```
>hqd:=stemnormal([op(hqc),op(comhqcs[2]),switch=[algebraic,factor]]:
```

```
    Obtain algebraic equation with length = 11907
and containing {E11, E12, E22, P1, P2, P3, mu, omega, p, q1, q2, r2, theta, u1, u2, z1, z2}
    Substitute this algebraic equation into the system? (y,n) n
    Obtain algebraic equation with length = 4748
and containing {E11, E12, E22, P1, P2, P3, mu, omega, p, q1, r2, theta, u1, u2, z1, z2}
    Substitute this algebraic equation into the system? (y,n) n
    Obtain algebraic equation with length = 19486
and containing {E11, E12, E22, P1, P2, P3, P4, mu, omega, p, q1, r1, r2, theta, u1, u2, z1, z2}
    Substitute this algebraic equation into the system? (y,n) n
    Obtain algebraic equation with length = 5914
and containing {E11, E12, E22, P1, P2, P3, mu, omega, p, q1, r2, theta, u1, u2, z1, z2}
    Substitute this algebraic equation into the system? (y,n) n
```

```
>addpolsa:=sortl([op(getUnusedEqn()),op(pola),op(comres)]):maplen(%);
```

[1341, 1357, 4748, 5914, 10405, 11907, 13363, 13985, 14433, 14433, 19486, 19520]

Before proceeding with yet another application of *findComm*, it is important that the *clearall* command be executed prior to carrying out the automated commutation process. This will then clear the memory tables of previous assumptions, automated commutators, stored polynomials in *getUnusedEqn()*, etc so that only the currently relevant assumptions appear. Of course this may lead to redundant computations. However, it is very important to bear in mind that if a system is altered by injection of new conditions, or subjected to further normalisations/reductions, carrying out the same commutators as done previously can lead to new interesting results! One final set of commutators is carried out and the polynomial conditions stored in *commzu*.

```
>commzu:=stemreduce(map(autocomm,[z1,z2,u2]),hqd):comz12u2:=sortl(clean(map(foso,[seq(op(commzu[i]),i=1..3)]))):
>maplen(%);
```

[1354, 10402, 13360, 13982, 14430, 14430, 19517]

It is now time to switch to the basic variables, as given by VdBS, together with the scaled operators, followed by a complete reduction/normalisation of the combined differential system. It turns out that replacing p by $x \equiv p + \mu$ is indeed very useful as there is a remarkable reduction in overall system size.

```
>l:=lambda:R:=0:b3:=0:D3:=0:R3:=0:C13:=0:C23:=0:U3:=0:e3(1):=0:q3:=0:r3:=0:
>basicconv:=clean(eval(eqns(87,104))):assign(clean(eval(eqns(150,153)))):
>assign(clean(eval(eqns(146,148)))):basicvars:=[D0,D1,D2,R,b1,b2,R1,R2,J,C0,C3,C12,seq(X||i,i=1..16)]:
>for i from 1 to 28 do e0(basicvars[i]):=0:od:G := -(1/3)*(3*P1^2-3*P2*mu-3*P2*p-P1)/P1:
>p:=x-mu:e0(x) := -theta*x*(P1+1):e1(x):= -U1*lambda*x*(P1+1):e2(x) := -U2*lambda*x*(P1+1):
>e3(x) := -U3*lambda*x*(P1+1):modbasicv:=subsop(8=NULL,basicconv):
>assign(solve(modbasicv,{W,z1,z2,q1,q2,r1,r2,E11,E22,E12})):icona:=factor(basicconv[8]):
>C13:=0:C23:=0:R3:=0:D3:=0:R2:=0:D1:=0:
>addpolsc:=clean(map(numer,map(foso,clean(sortl(factor([op(addpolsa),op(comz12u2),icona)]))))) :
>maplen(addpolsc);
```

[389, 1234, 1234, 1239, 5293, 6201, 7363, 7363, 8518, 10324, 10324, 10402, 10402,...,14405, 14400, 14400]

```
>vars:=[C0,C12,C3,D0,D2,K3,J,P1,P2,P3,P4,R1,U1,U2,b1,b2,l,M,H,x,seq(X||i,i=1..16)]:clearall();
>for i from 1 to 36 do e3(vars[i]):=0:od:
```

Next, the complete differential system is normalised

```
>hqe:=stemnormal([op(hqc),op(eqns(105,135))],switch=[algebraic,factor]):
```

```

factor, 1, :, λ
factor , 2, :, 10D0P1U2λ-3P1b1λ-10u2D0+6e2(D0)P1
Use factor number?(ter=terminate) 2
factor, 1, :, D0
factor, 2, :, 162D0P12U1λ5-36D0P1U1λ5+2D0U1λ5-27P1b2λ5+3b2λ5+
36P12θU2x-36P2θU2x2-36P2e0(μ)U2x-4P1θU2x-36e0(U2)P1x+4e0(U2)x
Use factor number?(ter=terminate) 2
"Obtain algebraic equation"

```

3*C3-2*J

```

Substitute this algebraic equation into the system? (y,n) y
"Obtain algebraic equation"

```

C12

```

Substitute this algebraic equation into the system? (y,n) y
"Obtain algebraic equation"
27*D02*P1*lambda10-15*D02*lambda10+6*P12*U12*lambda2*x2+
6*P12*U22*lambda2*x2-6*P2*U12*lambda2*x3-6*P2*U22*lambda2*x3-
5*P1*U12*lambda2*x2-5*P1*U22*lambda2*x2-3*J*P1*lambda2*x2-
U12*lambda2*x2-U22*lambda2*x2+3*J*lambda2*x2+3*P1*theta2*x2-
9*P1*mu*x2+theta2*x2-3*mu*x2
Substitute this algebraic equation into the system? (y,n) n
Obtain algebraic equation with length = 1216
and containing {D0, J, P1, P2, P3, U1, U2, b1, b2, lambda, theta, x}
Substitute this algebraic equation into the system? (y,n) n
Obtain algebraic equation with length = 1890
and containing {D0, J, P1, P2, P3, U1, U2, X3, X5, b1, b2, lambda, mu, theta, x}
Substitute this algebraic equation into the system? (y,n) n
Obtain algebraic equation with length = 1255
and containing {D0, J, P1, P2, P3, U1, U2, X4, b1, b2, lambda, theta, x}
Substitute this algebraic equation into the system? (y,n) n
Obtain algebraic equation with length = 1890
and containing {D0, J, P1, P2, P3, U1, U2, X3, X5, b1, b2, lambda, mu, theta, x}
Substitute this algebraic equation into the system? (y,n) n
Obtain algebraic equation with length = 1606
and containing {D0, J, P1, P2, P3, U1, U2, X15, b1, b2, lambda, theta, x}
Substitute this algebraic equation into the system? (y,n) n
Obtain algebraic equation with length = 1606
and containing {D0, J, P1, P2, P3, U1, U2, X14, b1, b2, lambda, theta, x}
Substitute this algebraic equation into the system? (y,n) n
"Obtain algebraic equation"

```

2*X13+X14

```

Substitute this algebraic equation into the system? (y,n) y
Obtain algebraic equation with length = 1267
and containing {D0, J, P1, P2, P3, U1, U2, X13, b2, lambda, theta, x}
Substitute this algebraic equation into the system? (y,n) n
"Obtain algebraic equation"

```

2*X12-X15

```

Substitute this algebraic equation into the system? (y,n) y
Obtain algebraic equation with length = 1603
and containing {D0, J, P1, P2, P3, U1, U2, X12, b1, b2, lambda, theta, x}
Substitute this algebraic equation into the system? (y,n) n

```

>showAssumption();

Non-zero factors
 $D0 \neq 0$
 $\lambda \neq 0$
Non-zero denominators
 $x \neq 0$
 $P1 \neq 0$
 $U1 \neq 0$
 $U2 \neq 0$
 $\lambda \neq 0$
 $3 P1 + 1 \neq 0$
 $9 P1 - 1 \neq 0$
 $3P1^2 - 3P2x - P1 \neq 0$

```
>X10:=0:X9:=0:X16:=0:X1:=0:X2:=0:X7:=0:X8:=0:X14:=-2*X13:X15:=2*X12:X11:=0:C12:=0:J:=3/2*C3:
>xpolb:=sortl(factor([op(addpolsc),op(getUnusedEqn())])):
>clearall();hqe:=stemnormal(hqe,switch=[algebraic,factor]):
```

A few key points need to be made here. The user should take note of the particular choices made on which conditions to substitute back into the normalisation procedure and which are to be kept aside. Note factor choices *must* be made. Intuitively, one can substitute in conditions that are "relatively small" perhaps less than 1K Maple length but the user should be aware that in making each choice it is important to consider the actual structure of the condition, like high degrees in the variables which can adversely affect the automated process. A simple check shows that in the normalisation process we have internally generated two important polynomial conditions.

```
>hqe[1..2];
```

$[C0 = 0, X6 = 0]$

```
>C0:=0:X6:=0:
```

We can now use the available group freedom of a basic rotation in the $\langle e_1, e_2 \rangle$ blade to set $b_1 = 0$ without altering any of the conditions already established.

```
>b1:=0:
```

Next, we again normalise the system (this should be done since we have globally assigned the b_1 to zero) and using *minimisePoly* simplify/reduce the complete set of polynomials.

```
>clearall();hqe:=sortl(stemnormal(hqe,switch=[algebraic,factor])):
```

factor , 1, : , λ
factor , 2, : , $2R1b2 + X3 - X5$
Use factor number?(ter=terminate) 2

```
>fiso(2*R1*b2+X3-X5,X5):assign(%);ypola:=minimisePoly(xpolb):
>maplen(%);
```

[376, 560, 1022, 1030, 1057, 1170, 1272, 1359, 1411, 1437, 2710, 4740, 6268, 7452, 7573]

It is important to constantly reduce the differential system modulo the polynomial one when appropriate, as done below. This can have a beneficial affect of managing its overall size. Hence, we simplify/reduce the differential system using the polynomial system and then to follow it up with a normalisation together with an application of *minimisePoly* to the accrued complete set of polynomials.

```
>hqe:=stemnormal(simplifySys(hqe,ypola),switch=[algebraic,factor]):
>hqeb:=stemcomplete(hqe,switch=[algebraic,factor]):
```

Obtain algebraic equation with length = 543
and containing {C3, D0, P1, P2, U1, U2, X3, b2, lambda, x}
Substitute this algebraic equation into the system? (y,n) n
Obtain algebraic equation with length = 1499
and containing {C3, D0, P1, P2, P3, U1, U2, X12, b2, lambda, theta, x}
Substitute this algebraic equation into the system? (y,n) n
Obtain algebraic equation with length = 1536
and containing {C3, D0, P1, P2, P3, U1, U2, X13, b2, lambda, theta, x}
Substitute this algebraic equation into the system? (y,n) n

•
•
•

```
>ypolb:=minimisePoly([op(getUnusedEqn()),op(ypola)]:ypolb[1];
```

$$(3P1+1)(2D2b2+X4)\lambda P1x$$

```
>fiso(2*D2*b2+X4,X4):assign(%);ypolb:=minimisePoly(ypolb):
```

Again, we apply *findComm* to *hgeb* for the automated process of carry out all possible commutators

```
>clearall();comhgeb:=findComm(hgeb):maplen(%);
```

[1063, 1136, 1357, 520, 1492, 504, 1529, 537, 1437]

```
>showAssumption();
```

Non-zero factors

$$x \neq 0$$

$$P1 \neq 0$$

$$\lambda \neq 0$$

$$3 P1 + 1 \neq 0$$

Non-zero denominators

$$x \neq 0$$

$$b2 \neq 0$$

$$\lambda \neq 0$$

$$3 P1 - 1 \neq 0$$

$$3 P1 + 1 \neq 0$$

We note that stem has assumed $b_2 \neq 0$, in the above process. This observation is quite important in its relevance to the proof at the end. The next step consists of propagation in all directions of some (smaller) accrued polynomials. This complete set is simplified/reduced using *simplifySys* and is then split between polynomial and differential conditions with the augmentation of appropriate lists followed by subsequent normalisation, using *stemnormal* and reduction using *minimisePoly*

```
>hota:=minimisePoly(clean([op(ypolb),op(comhgeb)])):
```

```
>d0hota:=sortl(stemreduce(map(e0,hota),hgeb))[1..13]:
```

```
>d1hota:=stemreduce(map(e1,hota[1..16]),hgeb)[1..11]:
```

```
>d2hota:=stemreduce(map(e2,hota[1..16]),hgeb)[1..11]:
```

```
>totpropk:=sortl([op(d0hota),op(d1hota),op(d2hota)]):
```

```
>redtots:=separate(simplifySys(totpropk,hota)):
```

```
>maplen(redtots[1]);
```

[1010, 1217, 2519, 3121, 5310, 5991, 6591, 6806, 6813, 6817, 7418, 7709, 8058, 8293, 8685, 13630, 15465]

```
>maplen(redtots[2]);
```

[2324, 2436, 3016, 3338, 3398, 3486, 5732, 5986, 6402, 6460, 6472, 7041, 7558, 7580, 7599, 7633]

```
>clearall();hqh:=stemnormal([op(hgeb),op(redtots[2])],switch=[algebraic,factor]):
```

factor , 1, : , λ

$$\text{factor ,2, : , } -594P1^3U2^2b2\lambda^2x^2-54P1^2U2^2b2\lambda^2x^2+66P1U2^2b2\lambda^2x^2+36b2x^3...$$

Use factor number?(ter=terminate) 2

Obtain algebraic equation with length = 6824

and containing {C3, D0, D2, P1, P2, P3, P4, R1, U1, U2, X12, b2, lambda, theta, x}

Substitute this algebraic equation into the system? (y,n) n

Obtain algebraic equation with length = 8293

and containing {C3, D0, D2, P1, P2, P3, P4, R1, U1, U2, X12, X13, X3, b2, lambda, theta, x}

Substitute this algebraic equation into the system? (y,n) n

•
•
•

```
>hotb:=sortl(remredun(sortl([op(hota),op(getUnusedEqn()),op(redtots[1])]))):
```

```
>specs(hotb);
```

indets = {C3, D0, D2, P1, P2, P3, P4, R1, U1, U2, X12, X13, X3, b2, λ , μ , θ , x}

length = 221419

nops = 42

type = list

>hqi:=stemnormal(simplifySys(hqh,hotb),switch=[algebraic,factor]):

The next few steps should be self-explanatory as they are just more of the same as carried out above with the exception of *remfacs* which removes designated factors from a list of polynomials

```
>coma:=stemreduce(map(autocomm,[U1,U2,M,H]),hqi):
>comb:=stemreduce([comm(e1,e2,C3),comm(e1,e2,D0),comm(e1,e2,D2),comm(e1,e2,R1),
comm(e1,e2,X12),comm(e1,e2,X13),comm(e1,e2,X3),comm(e1,e2,b2)],hqi):
>comh:=map(numer,map(foso,clea(sortl([seq(op(coma[i]),i=1..4),op(comb)))])):
>hotc:=map(numer,sortl([op(hotb),op(comh)])[1..55]):
>specs(hotc);
```

```
indets = {C3, D0, D2, P1, P2, P3, P4, R1, U1, U2, X12, X13, X3, b2, λ, μ, θ, x}
length = 273843
nops = 55
type = list
```

```
>hotd:=remfacs(hotc,[seq(x^i,i=1..6),seq(1^i,i=1..7),seq((3*P1+1)^i,i=1..8),P1]):
>hotda:=minimisePoly(hotd,list):hote:=remfacs(hotda,[D0,l,x,3*P1+1,36*P1^2-27*P2*x+33*P1+13]):
>hote:=minimisePoly(hote):
>maplen(%):
```

```
[376, 502, 518, 534, 823, 1004, 1022, 1065, 1138, 1143, 1217, 1338, 1339, 1359, 1404,...,13442, 14467]
```

```
>d0hote:=sortl(stemreduce(map(e0,hote[1..28]),hqi))[1..20]:d1hote:=clean(stemreduce(map(e1,hote[1..12]),hqi)):
>d2hote:=sortl(clean(stemreduce(map(e2,hote[1..11]),hqi))):propm:=sortl([op(d0hote),op(d1hote),op(d2hote)]):
>redpropm:=simplifySys(propm,hote):hotf:=sortl([op(hote),op(redpropm)]):
```

We are now able to obtain expressions for U_1^2 , θ^2 , and θ .

```
>solUU1:=fiso(hotf[1],U1^2);
```

$$\text{solUU1} := U_1^2 = (1/2)(-54D0^2P1\lambda^{10}+30D0^2\lambda^{10}-12P1^2U2^2\lambda^2x^2+12P2U2^2\lambda^2x^3+10P1U2^2\lambda^2x^2+9C3P1\lambda^2x^2+2U2^2\lambda^2x^2-9C3\lambda^2x^2-6P1\theta^2x^2+18P1\mu x^2-2\theta^2x^2+6\mu x^2)/(\lambda^2x^2(6P1^2-6P2x-5P1-1))$$

```
>solHH:=fiso(foso(powsubs(solUU1,hotf[4])),H^2);
```

$$\begin{aligned} \text{solHH} := \theta^2 = & -(3/2)(-90D0^3P1^2\lambda^{10}+144D0^3P2\lambda^{10}x+144D0^3P1\lambda^{10}-6D0^3\lambda^{10}+24P1^3U1b2\lambda^2x^2- \\ & 24P1P2U1b2\lambda^2x^3+27C3D0P1^2\lambda^2x^2-36C3D0P2\lambda^2x^3-26P1^2U1b2\lambda^2x^2+ \\ & 6P2U1b2\lambda^2x^3-30C3D0P1\lambda^2x^2+6P1^2X3\lambda^2x^2+P1U1b2\lambda^2x^2-6P2X3\lambda^2x^3+3C3D0\lambda^2x^2- \\ & 66D0P1^2\mu x^2+24D0P1^2x^3+48D0P2\mu x^3-24D0P2x^4-45P1X3\lambda^2x^2+U1b2\lambda^2x^2+16D0P1\mu x^2- \\ & 20D0P1x^3-X3\lambda^2x^2+2D0\mu x^2-4D0x^3)/(D0x^2(33P1^2-24P2x-8P1-1)) \end{aligned}$$

```
>solH:=fiso(factor(powsubs(solUU1,solHH,hotf[5])),H);
```

$$\begin{aligned} \text{solH} := \theta = & (3/2)U2b2\lambda^7x D0(33P1^2-24P2x-8P1-1)(6P1^2-6P2x-5P1-1)/(-81D0^3P1^4\lambda^{10}+1296D0^3P1^2P2\lambda^{10}x- \\ & 1296D0^3P1P3\lambda^{10}x^2+1296D0^3P2^2\lambda^{10}x^2-108D0^3P1^3\lambda^{10}-1224D0^3P1P2\lambda^{10}x+144D0^3P3\lambda^{10}x^2+600D0^3P1^2\lambda^{10}- \\ & 120D0^3P2\lambda^{10}x-116D0^3P1\lambda^{10}-...-4D0P1x^3) \end{aligned}$$

At last we have arrived at a critical condition, being an equation determining $P3$. Using it, we may simplify a number of polynomials.

```
>solP3:=fiso(numer(factor(powsubs(solUU1,solH,stemreduce(e0(solUU1),hqi)))/(-3*(U2*b2)*(9*P2*x-2))/1^5/x^2,P3));
```

$$\text{solP3} := P3 = -(1/9)(54P1^4-54P1^2P2x-27P2^2x^2+9P1^3-9P1P2x-6P1^2+9P2x-P1)/(x^2(3P1+1))$$

```
>solP4:=fiso(fsubs(solP3,stemreduce(e0(solP3),hqi)),P4);
```

$$\begin{aligned} \text{solP4} := P4 = & (1/9)(-648P1^4P2x+648P1^2P2^2x^2+81P2^3x^3+378P1^5-459P1^3P2x+189P1P2^2x^2+225P1^4- \\ & 135P1^2P2x-45P2^2x^2-15P1^3+27P1P2x-25P1^2+19P2x-3P1)/(x^3(3P1+1)^2) \end{aligned}$$

```
>molH:=fsubs(solP3,solH);
```

$$\begin{aligned} \text{molH} := \theta = & -(3/4)(3P1+1)(6P1^2-6P2x-5P1-1)U2b2\lambda^5x/(-81D0^2P1^3\lambda^8-9D0^2P1^2\lambda^8+36D0^2P2\lambda^8x- \\ & 15D0^2P1\lambda^8-7D0^2\lambda^8+9C3P1^2x^2-9C3P2x^3+3C3P1x^2) \end{aligned}$$

```
>hotg:=minimisePoly(sortl(hotf)):solHH:=fiso(hotg[1],H^2);
```

$$\begin{aligned} \text{solHH} := \theta^2 = & (1/2)(-54D0^2P1\lambda^{10}+30D0^2\lambda^{10}-12P1^2U1^2\lambda^2x^2-12P1^2U2^2\lambda^2x^2+12P2U1^2\lambda^2x^3+12P2U2^2\lambda^2x^3+ \\ & 10P1U1^2\lambda^2x^2+10P1U2^2\lambda^2x^2+9C3P1\lambda^2x^2+2U1^2\lambda^2x^2+2U2^2\lambda^2x^2-9C3\lambda^2x^2+18P1\mu x^2+6\mu x^2)/(x^2(3P1+1)) \end{aligned}$$

```
>hoth:=minimisePoly(remfacs(hotg,[P2,seq(l^i,i=1..10),seq(x^i,i=1..10),seq((3*P1+1)^i,i=1..10),b2,
-36*D0^2*lambda^8-9*P2*U1^2*x^3-9*P2*U2^2*x^3+2*U1^2*x^2+2*U2^2*x^2+9*C3*x^2])):
>specs(hoth);
```

```
indets = {C3, D0, D2, P1, P2, P3, P4, R1, U1, U2, X12, X13, X3, b2, λ, μ, θ, x}
length = 349015
nops = 61
type = list
```

```
>hqj:=stemnormal(simplifySys(hqi,hoth),switch=[algebraic,factor]):
```

The next two steps consist of a conceptually novel application of the use of resultants. Normally resultants are used at the very end stages of a particular mathematical computational scheme. In our case, we use it as a starting point for the final analysis branch so to speak. We systematically carry out all resultants on pairs of polynomials relative to a designated list of variables. This is an exhaustive measure with the aim of attempting to "separate" the variables which can be particularly useful when one is dealing with a large number of them. This is one of the key elements that has "unlocked" this problem in our approach. The issue here is though, that this does indeed produce many equations, as can be seen below. Typically, as a consequence, considerably redundancy is generated, so that it is desirable to have routines that conveniently and expeditiously remove "trivial" conditions like simple constant multiples, repeated redundant factors, reducible equations, etc. To assist the user with this task, we have introduced a number of routines suitable for this purpose, like *remredun*, *remfacs*, *indeppol*, *simplifySys*, etc. Note that *genreslist* computes the resultant using all possible pairs of polynomials with respect to each of the given variables, as long as both polynomials have that variable, of any degree, present. On the other hand *linreslist* only selects pairs of polynomials which are linear in each of the given variables. The starting point and maximum size of each resultant which is to be retained, is controlled by the user.

```
>reshoth:=linreslist([C3, D0, D2, P1, P2, P3, P4, R1, U1, U2, X12, X13, X3, b2, l, M, H, x],1,7000,2500,hoth):
>modres:=factor(indeppol(reshoth,[],hoth)):
>specs(%);
```

```
indets = {C3, D0, D2, P1, P2, P3, R1, U1, U2, X12, X13, X3, b2, λ, μ, θ, x}
length = 241078
nops = 145
type = list
```

```
>hoti:=sortl([op(hoth),op(modresa)]):
>reshoti:=linreslist([C3, D0, D2, P1, P2, P3, P4, R1, U1, U2, X12, X13, X3, b2, lambda, mu, theta, x],1,5000,1500,hoti):
>resj:=indeppol(factor(reshoti,[],hoti)):
>hotj:=remredun(remfacs(sortl([op(hoti),op(resj)]),[seq(D0^i,i=1..4),seq(l^i,i=1..20),seq(x^i,i=1..20),
U1^2+U2^2,H,H^2,15*P1+8,seq(P1^i,i=1..10),U1,U2,U3,seq((3*P1+1)^i,i=1..6),198*P1^2+42*P1+5,
(U1^2+U2^2)^2,P2,54*P1^4-54*P1^2*P2*x+27*P1^3*P2*x^2-27*P2^2*x^2+9*P1^3-9*P1^2*x+
9*P3*x^2-6*P1^2+9*P2*x-P1,153*P1^2+78*P1+37]))):
>hotk:=indeppol(hotj,[],hotj[1..5]):
>specs(hotk);
```

```
indets = {C3, D0, D2, P1, P2, P3, P4, R1, U1, U2, X12, X13, X3, b2, λ, μ, θ, x}
length = 701027
nops = 309
type = list
```

We see that after the generation/reduction processes we are left with 309 conditions with the indicated variables. Next, we take the first 124 polynomials substitute in the expressions for p''' , p'''' , and U_1^2 and apply *minimisePoly* twice to obtain *eqb*

```
>eqa:=minimisePoly(map(numer,factor(powsubs(solP3,solP4,solUU1,hotj[1..124])))):
>eqb:=minimisePoly(factor(powsubs(solUU1,eqa))):
>maplen(%);
```

```
[297, 421, 440, 502, 518, 564, 797, 1384, 1418, 1531, 1588, 1624, 1720, 1751, 1846, 2001, 2277, 4484, 9483]
```

Fortuitously, the first conditions yields an expression for U_2 and subsequently one for U_1 .

```
>solU2:=fiso(molH,U2);
```

$$\text{solU2} := U_2 = -(4/3)\theta(-81D0^2P1^3\lambda^8-9D0^2P1^2\lambda^8+36D0^2P2\lambda^8x-15D0^2P1\lambda^8-7D0^2\lambda^8+9C3P1^2x^2-9C3P2x^3+3C3P1x^2)/((3P1+1)(6P1^2-6P2x-5P1-1)b2\lambda^5x)$$

```
>eqc:=sortl(clean(map(numer,fsubs(solU2,eqb))));
>solU1:=fiso(eqc[1],U1);
```

$$\begin{aligned} \text{solU1} := U1 = & -(1/3)(-270D0^3P1^2\lambda^{10}+432D0^3P2\lambda^{10}x+432D0^3P1\lambda^{10}-18D0^3\lambda^{10}+81C3D0P1^2\lambda^2x^2-108C3D0P2\lambda^2x^3- \\ & 90C3D0P1\lambda^2x^2+66D0P1^2\theta^2x^2-48D0P2\theta^2x^3+18P1^2X3\lambda^2x^2-18P2X3\lambda^2x^3+9C3D0\lambda^2x^2-198D0P1^2\mu x^2+ \\ & 72D0P1^2x^3-16D0P1\theta^2x^2+144D0P2\mu x^3-72D0P2x^4-15P1X3\lambda^2x^2+48D0P1\mu x^2-60D0P1x^3-2D0\theta^2x^2- \\ & 3X3\lambda^2x^2+6D0\mu x^2-12D0x^3)/(b2\lambda^2x^2(4P1-1)(6P1^2-6P2x-5P1-1)) \end{aligned}$$

Finally, we are now in a position to eliminate U_1 using the above and θ using resultants ($fres:=factor@resultant$), combined with one time propagation.

```
>eqd:=remredun(sortl(clean(map(numer,fsubs(solU1,eqc)))))
>eqe:=remfacs(eqd,[l,H,3*P1+1]):eqe:=minimisePoly(eqe):
>fin1:=sqrt(op(6,fres(eqc[1],eqe[2],H)),symbolic):
>fin2:=numer(fsubs(solP3,stemreduce(e0(fin1),hqj))/H):
```

Both $fin1$ and $fin2$ contain the variables $\{C_3, D_0, p', p'', X_3, \lambda, x\}$, so we choose to first eliminate C_3 and then X_3 . This is the best option as we know all derivatives of D_0 , and they are simple, being zero except for $e_1(D_0) = -(1/2)b_2\lambda$, which is nonzero since $b_2 \neq 0$, assumed above.

```
>grand:=fres(fin1,fin2,C3)/(18*D0*lambda^2*x^2):
>d0grand:=op(3,fsubs(solP3,stemreduce(e0(grand),hqj))):
>grand1:=fres(fin1,d0grand,C3):
>maxa:=-fres(grand1,grand,X3)/(2*D0*lambda^2*x^2*(3*P1+1))/(3*P1^2-3*P2*x-P1):
>specsop(%);
```

```
1
indets = {P1, P2, x}
length = 280
nops = 17
type = +
2
indets = {D0, P1, P2, lambda, x}
length = 10717
nops = 292
type = +
```

```
>scon:=map(factor,collect(op(1,maxa),[P2,P1],factor));
```

$$\begin{aligned} \text{scon} := & 54x^3(45P1-37)P2^3-9x^2(756P1^3-459P1^2-158P1+33)P2^2+3xP1(1134P1^4-189P1^3- \\ & 975P1^2+227P1+19)P2+2P1^2(3P1-1)(162P1^4-45P1^3+69P1^2+16P1-10) \end{aligned}$$

```
>
>fin3:=op(3,maxa):alldeg(%);
```

$$D0 = 2, P1 = 25, P2 = 10, \lambda = 10, x = 12$$

Now $fin3$ is a depressed quadratic in D_0 which is not constant. Hence the coefficients of D_0^0 and D_0^2 of $fin3$ must vanish.

```
>cof0:=fcoeff(fin,D0,0)/(-5*x^3*(3*P1+1))/(3*P1^2-3*P2*x-P1):specs(%);
```

```
indets = {P1, P2, x}
length = 2487
nops = 102
type = +
```

```
>cof2:=fcoeff(fin,D0,2)/4/1^10:specs(%):
```

```
indets = {P1, P2, x}
length = 4687
nops = 163
type = +
```

```
>fres(cof0,cof2,P2)/x^80:
>specs(%);
```

```

indets = {P1}
length = 8160
nops = 9
type = *

```

Hence $p'=\text{constant}$ and the generic branch is done: There remain a few special cases like $b_2 = 0$, $\text{scon}=0$, $\text{U1U2}=0$, or $3*P1^2-3*P2*x-P1=0$ etc as appropriate at various stages in the above computation. These are all quite easily rejected but will not be dealt here due to space limitations.

Several final points should now be made. Firstly, many of the above steps concern themselves with diagnostics. One should liberally use certain routines like *specs*, *maplen (with sortl)*, *showAssumption*, *showCommutator*, etc. so as to carefully monitor the general progression of the overall computation. Secondly, we stress that the generation/reduction processes are highly "non-linear" and hence final (and intermediate) system sizes and structures depend very much on intermediate steps and the order in which they are carried out. Also, the variable ordering made at the beginning is important in the overall scheme. Thirdly, at certain points, the user needs to decide which conditions to substitute back into the normalizing processes and which ones to keep aside. There are no hard-and-fast rules here. Depending on the nature of the problem, individual users must decide, perhaps guided by some experimentation or "intuition", on how large a system they wish to retain. Without exercising care at critical points, the computer may be overwhelmed with intermediate swell problems coupled with unacceptable time/memory requirements.

5 Comments

Typically, as seen in the examples above, in general relativity one usually encounters large complex systems of equations. Hence, to assist the user in the investigation of such systems, *STeM* provides interactive tools to help make important decisions at certain critical points in carrying out the analysis process. We emphasize that frequently, the best approach, which fundamentally relies on the ordering of the variables that affect *STeM*'s reduction routines, is not immediately obvious and therefore some experimentation may be required.

In assembling the different parts of *STeM*, we have been required to input many hundreds of equations from various sources. Clearly, the challenge has been to provide consistent, in the sense of signature, duals and object definitions, systems of equations without typographical error. We have done our best to try and avoid errors by performing very extensive cross-checking but however being that this is the first release of *STeM*, we would advise the users to exercise a certain amount of caution regarding the veracity of all equations. Separate independent checks is advised. We hope that through the widespread use of this package any errors that may be present will be found. In such cases, we would much appreciate receiving notification of these issues so that they may be addressed. In the second part exposition of *STeM*, we will demonstrate by examining further applications, the functionality of the remaining routines that we have not covered here. As a final point, in the near future, if there is sufficient interest, we will develop a version of *STeM* for Mathematica.

6 Acknowledgments

First of all, we would like to thank N.Van den Bergh for providing us with most of the orthonormal equations that are in *STeM*. It is his notation and conventions, except for the tetrad rotation coefficients renaming, that we have implemented in the orthonormal part of our package. We would also thank him for providing the Maple worksheets containing the expressions of the defining basic variables and basic integrability conditions/EFE as in Appendix B of his paper, and for his continued encouragement and suggestions throughout various aspects of this work. For the NP and GHP Killing equations, with resulting integrability conditions, (these were independently verified by *STeM*) we thank J.D. Steele.

7 Appendix: Basic variables

$$\begin{aligned}
\mathfrak{D}_0 &= \frac{(p+\mu)\omega}{\lambda^5}, \mathfrak{R} = \frac{n}{\lambda}, \mathfrak{h}_1 = -\frac{4}{3} \frac{p+\mu}{\lambda^6} z_1 - \frac{2(9p'-1)(p+\mu)\omega}{3\lambda^5} \dot{U}_2, \\
\mathfrak{h}_2 &= -\frac{4}{3} \frac{p+\mu}{\lambda^6} z_2 + \frac{2(9p'-1)(p+\mu)\omega}{3\lambda^5} \dot{U}_1, \mathfrak{h}_3 = -\frac{4}{3} \frac{p+\mu}{\lambda^6} z_3, \\
\mathfrak{D}_1 &= -\frac{1}{3} \dot{U}_1 + \frac{q_1}{\lambda}, \mathfrak{R}_1 = \frac{1}{3} \dot{U}_1 + \frac{r_1}{\lambda}, \mathfrak{D}_2 = -\frac{1}{3} \dot{U}_2 + \frac{q_2}{\lambda}, \\
\mathfrak{R}_2 &= \frac{1}{3} \dot{U}_2 + \frac{r_2}{\lambda}, \mathfrak{D}_3 + \mathfrak{R}_3 = -\frac{1}{3} \dot{U}_3 + \frac{q_3}{\lambda}, \mathfrak{D}_3 - \mathfrak{R}_3 = \frac{1}{3} \dot{U}_3 + \frac{r_3}{\lambda} \\
\mathfrak{J} &= (1-2G)\dot{U}^2 + \lambda^{-2}(\theta^2 - 3\mu - 2\frac{j}{p'}) + 9\mathfrak{D}_0^2 \frac{\lambda^8}{(p+\mu)^2}, \\
\mathfrak{C}_{\alpha\beta} &= \frac{3p'+1}{\lambda^2 p'} E_{\alpha\beta} + G\dot{U}_\alpha \dot{U}_\beta, \quad (\alpha, \beta) = (1, 2), (1, 3), (2, 3), \\
\mathfrak{C}_0 &= \frac{3p'+1}{\lambda^2 p'} (E_{11} - E_{22}) + G(\dot{U}_1^2 - \dot{U}_2^2), \\
\mathfrak{C}_3 &= \frac{3p'+1}{\lambda^2 p'} E_{33} - \frac{G}{3} (\dot{U}_1^2 + \dot{U}_2^2 - 2\dot{U}_3^2) + \frac{2(9p'+1)\mathfrak{D}_0^2 \lambda^8}{3p'(p+\mu)^2}, \\
G &\equiv \frac{1}{3p'} \{3p''(p+\mu) + p' - 3(p')^2\}, \lambda \equiv \exp\left(\int \frac{d\mu}{3(p+\mu)}\right). \\
\dot{U}_\alpha &\equiv \frac{\dot{u}_\alpha}{\lambda p'}.
\end{aligned}$$

References

1. Banerji S. 1968 Prog. Theor. Phys. **39** 365
2. Carminati J 1987 J. Math. Phys. **28** 1848
3. Carminati, J. and Cyganowski S 1997 Class. Quantum Grav. **14**, 1167
4. Collins C B 1984 J. Math. Phys. **25** 995
5. Collins C B 1986 Can. J. Phys. **64**, 191
6. Coley, A. A. 1991 Class. Quantum Grav. **8** 995
7. Complete details of the calculations for all cases, are available in Maple worksheets from the authors
8. Cyganowski S and Carminati J 2000 Gen. Rel. Grav. **32** 221
9. Czapor S. R. and McLenaghan R. G. 1987 Gen. Rel. Grav. **19** 623
10. Ellis G F R (1971) *Relativistic Cosmology* R Sachs (Ed.), *General Relativity and Cosmology*, Rendiconti S. I. F., XLVIII Corso, Academic Press,. New York.
11. Ellis G F R 1967 J. Math. Phys. **8** 1171
12. Geroch R, Held A and Penrose R 1973 *JMP* **14** 874
13. King A. R. and Ellis G. F. R. 1973 Commun. Math. Phys. **31** 209
14. Lang J M and Collins C B 1988 Gen. Rel. Grav. **20** 683
15. MacCallum M A H 1971 *Cosmological Models from a Geometric Point of View*, in *Cargese Lectures in Physics*, Vol. 6, Lectures at the International Summer School of Physics, Cargese, Corsica, edited by E. Schatzman (Gordon and Breach, New York, 1973)
16. MacCallum, M.A.H. 2018 Computer Algebra in Gravity Research. Living Rev Relativ **21**, 6.
17. Newman E T and Penrose R 1962 *J. Math. Phys.* **3** 556
18. Senovilla J. M. M., Sopuerta C. F. and Szekeres P. 1998 Gen. Rel. Grav. **30** 389
19. Sopuerta C F 1998 Class. Quantum Grav. **15** 1043
20. Steele J D 2012 arXiv:1212.1179v1 [gr-qc] 5 Dec
21. Stephani H, Kramer D, MacCallum M A H, Hoenselaers C and Herlt E 2003 *Exact Solutions of Einstein's Field Equations* 2nd Ed. (Cambridge: Cambridge University Press)
22. The notation used is as in Vu K T and Carminati J 2003 Gen. Rel. Grav. **35**, 263
23. Treciokas R and Ellis G F R 1971 Commun. Math. Phys. **23** 1
24. Van den Bergh, N. 1987 Class Quantum Grav 5:L169–79
25. Van den Bergh N 1999 Class. Quantum Grav. **16** 117
26. Van den Bergh, N and Slobodeanu, R 2016 Class. Quantum Grav. **33**, 085008
27. Van den Bergh, N 2023 arXiv:2309.10478v3 [gr-qc] 12 Oct 2023
28. Vu, K T and Carminati J 2001 Gen. Rel. Grav. **33** 295
29. Vu, K T and Carminati J 2003 Gen. Rel. Grav. **35** 263
30. White A. J. and Collins C. B. 1984 J. Math. Phys. **25** 332